wt-orchestrate

Hogyan Működik --- A teljes orchestrációs pipeline

2026. március

wt-tools // autonomous orchestration framework

>_ built with Claude Code agents

Tartalomjegyzék

1 25 évvel később 6

2 Miért van erre szükség? 7
- 2.1 A probléma 7
- 2.2 A megoldás: autonóm orchestráció 7
- 2.3 Kinek szól? 8
- 2.4 Miben más, mint egy CI/CD rendszer? 8

3 Fejlődéstörténet 9

4 A wt-tools ökoszisztéma 10
- 4.1 Áttekintés 10
- 4.2 A fő eszközök 10
  - 4.2.1 Worktree lifecycle 10
  - 4.2.2 Ágens futtatás 10
  - 4.2.3 Orchestráció és felügyelet 11
  - 4.2.4 Memória és kontextus 11
  - 4.2.5 Projekt menedzsment 11
  - 4.2.6 Csapat szinkronizáció 12
  - 4.2.7 OpenSpec és minőség 12
- 4.3 Projekt sablonok 12
  - 4.3.1 wt-project-base 12
  - 4.3.2 wt-project-web (Next.js) 12
  - 4.3.3 Más irányok 13
- 4.4 Az ökoszisztéma térképe 13

5 Áttekintés 14
- 5.1 Mi az orchestráció? 14
- 5.2 Az 5-réteg modell 14
- 5.3 Moduláris architektúra 15
- 5.4 Állapotfájlok 15
- 5.5 Egy tipikus futás 17

6 Input és Konfiguráció 18
- 6.1 Input módok 18
  - 6.1.1 1. Specifikáció mód (`--spec`) 18
  - 6.1.2 2. Brief mód (`--brief`) 19
  - 6.1.3 3. Auto-detect 19
- 6.2 Konfiguráció 19
  - 6.2.1 orchestration.yaml 19
  - 6.2.2 Dokumentum direktívák 20
- 6.3 Direktíva referencia 20

7 Digest és Triage 22
- 7.1 Mi a Digest? 22
- 7.2 A Digest generálás lépései 22
  - 7.2.1 1. Spec szkennelés (`scan_spec_directory`) 22
  - 7.2.2 2. Prompt összeállítás (`build_digest_prompt`) 22
  - 7.2.3 3. API hívás és parsing 22
  - 7.2.4 4. ID stabilizálás (`stabilize_ids`) 24
- 7.3 A digest kimeneti fájljai 24

7.4 Frissesség ellenőrzés 24
7.5 Ambiguity Triage 24
7.5.1 Triage generálás 25
7.5.2 Emberi döntés 25
7.5.3 Automatikus triage mergelés 25
7.6 Requirement Coverage 25

8 OpenSpec --- A Spec-Driven Workflow 27
8.1 Mi az OpenSpec? 27
8.2 Az artifact-ok 27
8.2.1 Proposal 27
8.2.2 Design 27
8.2.3 Specs 27
8.2.4 Tasks 27
8.3 Integráció az orchestrációval 28
8.3.1 Dispatch fázis 28
8.3.2 Ralph loop integráció 28
8.3.3 FF→Apply láncolás 28
8.4 Interaktív vs. orchestrált használat 28
8.5 Nyomkövethetőség 29
8.6 Limitációk 29

9 Tervezés 30
9.1 A dekompozíció célja 30
9.2 Tervezési módszerek 30
9.2.1 API mód (alapértelmezett) 30
9.2.2 Agent mód 30
9.3 A Plan struktúra 32
9.3.1 Mezők magyarázata 32
9.4 Topológiai rendezés (DAG) 33
9.4.1 Körkörös függőség detekció 33
9.5 Tesztelési infrastruktúra felderítés 33
9.6 Spec összegzés nagy dokumentumokhoz 33
9.7 Plan validálás 34

10 Végrehajtás 35
10.1 Dispatch: a change elindul 35
10.1.11. Worktree létrehozás 35
10.1.22. Bootstrap 36
10.1.33. Szinkronizálás a fő ággal 36
10.1.44. Ralph Loop indítás 36
10.2 A Ralph Loop 36
10.2.1Iterációs ciklus 36
10.2.2loop-state.json 36
10.2.3Befejezési feltételek 37
10.2.4Context Pruning 37
10.3 Model Routing 37
10.3.1Complexity alapú routing 37
10.4 Párhuzamos végrehajtás 38
10.5 Pause és Resume 38
10.6 Token tracking 38

11 A Sentinel 39
11.1 Miért kell felügyelő a felügyelőnek? 39
11.2 Bash Sentinel 39
11.2.1Crash Recovery exponenciális backoff-fal 39
11.2.2Élőség-detekció 39
11.2.3Állapot-helyreállítás 40
11.2.4Kilépési osztályozás 40
11.3 Ágens Sentinel 40
11.3.1Tiered beavatkozás 40

11.3.2 Checkpoint kezelés 41
11.3.3 Autonómia szabályok 41
11.4 Sentinel események 41

12 Monitor és Watchdog 42
12.1 A Monitor Loop 42
12.1.1 A poll ciklus 42
12.1.2 poll_change() 42
12.1.3 Active time tracking 44
12.2 A Watchdog rendszer 44
12.2.1 Detekciós mechanizmusok 44
12.2.1.1 1. Timeout detekció 44
12.2.1.2 2. Action hash loop detekció 44
12.2.1.3 3. Artifact creation grace period 44
12.2.2 Eszkalációs szintek 46
12.2.3 Recovery detekció 46
12.3 Token biztonsági hálók 46
12.3.1 Soft limit (token_budget) 46
12.3.2 Hard limit (token_hard_limit) 46
12.4 Időlimit 47
12.5 Memória és auditálás 47

13 Minőségi Kapuk 48
13.1 Áttekintés 48
13.2 Test Gate 48
13.2.1 Működés 48
13.2.2 Paraméterek 48
13.2.3 Kimenet kezelés 49
13.3 Review Gate 49
13.3.1 Működés 49
13.3.2 Requirement-aware review 49
13.3.3 Konfiguráció 50
13.4 Verify Gate 50
13.4.1 Működés 50
13.4.2 Retry logika 50
13.5 Smoke Gate 50
13.5.1 Működés 50
13.5.2 Blocking vs Non-blocking 50
13.5.3 Smoke fix ciklus 51
13.5.4 Health Check 51
13.6 E2E Gate 51
13.6.1 Két mód 51
13.7 A gate-ek sorrendje összefoglalva 52
13.8 Hookak 52

14 Merge és Delivery 53
14.1 Merge Queue 53
14.2 Merge Policy 53
14.2.1 eager 53
14.2.2 checkpoint (alapértelmezett) 53
14.2.3 manual 53
14.3 A merge folyamat 55
14.3.1 Eset 1: Branch már nem létezik 55
14.3.2 Eset 2: Branch már be van mergelve 55
14.3.3 Eset 3: Normál merge (a gyakori) 55
14.4 3-rétegű Conflict Resolution 55
14.4.1 1. réteg: Generált fájlok (automatikus) 55
14.4.2 2. réteg: LLM merge (--llm-resolve) 55
14.4.3 3. réteg: Emberi beavatkozás 56
14.5 Post-Merge Pipeline 56
14.5.1 1. Futó worktree-k szinkronizálása 56

14.5.22. Base build ellenőrzés 56
14.5.33. Post-merge command 56
14.5.44. Change archiválás 56
14.5.55. Post-merge hook 56
14.5.66. Coverage frissítés 57
14.6 Checkpoint és Approval 57
14.6.1 Checkpoint aktiválás 57
14.6.2 Approval folyamat 57
14.6.3 Auto-approve (E2E futtatásokhoz) 57
14.7 Merge retry intelligencia 57

15 Tapasztalatok és kihívások 58
15.1 Megoldott problémák 58
15.1.1 Token budget kalibráció 58
15.1.2 Watchdog spam 58
15.1.3 Sentinel fájlnév eltérés 58
15.1.4 Token tracking félrevezető adatok 58
15.1.5 Nagy change-ek megbízhatósága 59
15.1.6 Jest + Playwright ütközés 59
15.2 Ismert limitációk 59
15.2.1 Mock-alapú tesztek elrejtik a runtime hibákat 59
15.2.2 Megosztott erőforrások merge konfliktusa 59
15.2.3 Kontextusvesztés merge után 59
15.2.4 Memória nem garantálja a minőséget 59
15.3 Ami még hátra van 60
15.3.1 Cascade failure logika 60
15.3.2 Trend-alapú token budget 60
15.3.3 Ágens-pontozás dispatch-hoz 60
15.3.4 Circuit breaker API hívásokhoz 60

16 Replan és Coverage 61
16.1 Auto-Replan 61
16.1.1 Aktiválás 61
16.1.2 Folyamat 61
16.1.3 Retry logika 61
16.1.4 All-done detekció 61
16.2 Manuális Replan 62
16.3 Requirement Coverage 62
16.3.1 Coverage riport 62
16.3.2 Coverage tracking lifecycle 63
16.3.3 Coverage gap detekció 63
16.3.4 Cross-cutting követelmények 63
16.4 Final Coverage Check 64
16.5 Összefoglaló riportok 64
16.5.1 HTML riport 64
16.5.2 Summary email 64
16.5.3 orchestration-summary.md 64

17 Hivatkozások 65
17.1 CLI Referencia 65
17.1.1 wt-orchestrate 65
17.1.1.1 Globális opciók 65
17.1.1.2 Parancsok 65
17.1.2 wt-new / wt-merge / wt-loop 66
17.2 Change állapotgép 66
17.2.1 Állapotok 67
17.2.2 Főbb állapotátmenetek 67
17.3 Direktíva táblázat 67
17.3.1 Végrehajtás 67
17.3.2 Modellek 68
17.3.3 Tesztelés 68

17.3.4 Review és Verify . . . 68
17.3.5 Token kontroll . . . 69
17.3.6 Tervezés . . . 69
17.3.7 Watchdog . . . 69
17.3.8 Hookak . . . 69
17.3.9 Eseménynapló . . . 70
17.4 Fájl struktúra referencia . . . 70
17.4.1 Projekt szintű fájlok . . . 70
17.4.2 Worktree szintű fájlok . . . 70
17.4.3 Eseménytípusok . . . 71

1. fejezet

25 évvel később

A 2000-es évek elején Magyarországon dolgoztam egy Nokia alvállalkozói csapatban. SMS gateway-eket és hasonló telekommunikációs rendszereket fejlesztettünk — a korszak nagy infrastruktúra projektjeit, ahol a megbízhatóság nem opció volt, hanem alapkövetelmény. Millió felhasználó üzenetei mentek át a szervereinken, és minden perc kiesés mért költséget jelentett. Ott tanultam meg, mit jelent a *rendszerszintű gondolkodás*: nem elég, ha egy komponens működik — az egész pipeline-nak kell működnie, végig, éjjel-nappal, emberi beavatkozás nélkül. Ha valami megállt, nem egy ember ment ránézni — a rendszer maga próbálta meg először javítani.

Huszonöt évvel később, egy teljesen más technológiai kontextusban, ugyanaz a minta köszönt vissza. A `wt-orchestrate` ugyanazt a problémát oldja meg: hogyan futtassunk *autonóm, több-lépéses pipeline-okat* úgy, hogy a rendszer magától kezelje a hibákat, magától eszkaláljon, és csak akkor kérjen embert, ha tényleg muszáj. Csak itt nem SMS-ek mennek át a pipeline-on, hanem *szoftver-fejlesztési feladatok*. Az "ágens" nem egy hardveres node, hanem egy Claude Code session egy git worktree-ben. A "watchdog" nem hardveres heartbeat-et figyel, hanem a `loop-state.json` iterációs hash-ét. De a minta ugyanaz:

- **Dispatch → Monitor → Detect failure → Escalate → Recover or fail gracefully**

Ez a dokumentáció végigvezet ezen a pipeline-on, az inputtól a végső merge-ig.

FONTOS

Ez nem egy elméleti dokumentum. Minden funkció, amit itt leírunk, éles projekteken lett tesztelve és fejlesztve — a wt-orchestrate naponta fut production kódbázisokon, autonóm módon, órákig, emberi beavatkozás nélkül.

2. fejezet

Miért van erre szükség?

2.1 A probléma

Az AI-alapú kódfejlesztő eszközök — mint a Claude Code, a Cursor, vagy a GitHub Copilot — forradalmasították az egyéni fejlesztői produktivitást. Egy fejlesztő egy session-ben képes órákon belül komplex feature-öket implementálni.

De mi van, ha a feladat **nem egy feature**, hanem *húsz*?

Egy közepes specifikáció (pl. egy SaaS alkalmazás következő verziója) tipikusan 10-30 önálló fejlesztési feladatot tartalmaz. Ezek között vannak függőségek: az autentikáció kell a profil oldalhoz, az API kell a dashboardhoz, a migrációk kellenek minden adatbázis-művelethez. Ha ezt kézzel menedzseljük:

- Egy-egy feature kész → kézzel merge, kézzel tesztelés
- Merge konfliktusok → kézzel feloldás, ismételt tesztelés
- Egy ágens elakad → kézzel debug, context-váltás, újraindítás
- Specifikáció változik → kézzel újratervezés, átprioritizálás
- Token budget elfogy → kézzel monitorozás, session újraindítás

Mindez **menedzsment overhead**: a fejlesztő idejének jelentős része nem kódolásra megy, hanem pipeline koordinációra.

2.2 A megoldás: autonóm orchestráció

A `wt-orchestrate` ezt az overheadet automatizálja. Egyetlen specifikációból kiindulva:

1. **Automatikusan megtervezi** a feladatokat (dekompozíció, DAG)
2. **Párhuzamosan végrehajtja** őket izolált worktree-kben
3. **Folyamatosan ellenőrzi** az ágenseket (15s poll, watchdog)
4. **Automatikusan teszteli és review-zza** a kész munkát (quality gate-ek)
5. **Összefésüli** az eredményt (merge, conflict resolution)
6. **Folytatja** a következő fázissal (auto-replan)

Ez azt jelenti, hogy egy specifikáció átadása után a rendszer akár *órákig* dolgozhat felügyelet nélkül, miközben a fejlesztő más feladatokkal foglalkozik — vagy alszik.

2.3 Kinek szól?

Ez a rendszer **egyéni fejlesztőknek és kis csapatoknak** készült, akik AI ágensekkel dolgoznak. Nem helyettesíti a CI/CD-t (Jenkins, GitHub Actions) — kiegészíti, a *fejlesztési fázisban*, mielőtt a kód egyáltalán a CI pipeline-ba kerülne.

Tipikus felhasználási minta:

```
Reggel:   Spec átadása → wt-orchestrate plan → start
Napközben: Más feladatok, meeting-ek, tervezés
Este:     wt-orchestrate status → 12/15 change merged, 2 running, 1 failed
          Review → approve → a maradék megy tovább
Másnap:   Minden kész, PR nyitása
```

FONTOS

A cél nem az, hogy az ember ”ne kelljen”. A cél az, hogy az ember ott kelljen, ahol az ember pótolhatatlan: a specifikáció írásánál, a tervezésnél, és a végső döntéseknél — nem a pipeline babysitting-jénél.

2.4 Miben más, mint egy CI/CD rendszer?

Szempont	CI/CD (pl. GitHub Actions)	wt-orchestrate
Mikor fut	Commit/PR után	Commit *előtt* --- a fejlesztés közben
Mit csinál	Build, test, deploy	Tervezés, implementáció, teszt, merge
Ki dolgozik	Determinisztikus scriptek	AI ágensek (kreatív, adaptív)
Hiba kezelés	Fail → piros pipeline	Fail → retry, redispatch, escalate
Párhuzamosság	Job matrix (fix)	DAG-alapú, dinamikus dispatch
Feedback loop	Ember javít, újra push	Ágens javít, újra tesztel

A kettő kiegészíti egymást: a `wt-orchestrate` előállítja a kódot, a CI/CD pipeline validálja és deployolja.

3. fejezet

Fejlődéstörténet

A `wt-tools` első commitja 2026. január 9-re datálódik — ekkor még egy belső, privát repóban élt, egyszerű worktree-kezelőként. A publikus fejlesztés februárban indult újra (a korábbi history force push miatt nem látszik a git log-ban). Január-februárban alakult ki a worktree lifecycle (`wt-new`, `wt-merge`, `wt-close`) és a Ralph iteratív loop. Februárban lett nyilvánvaló, hogy a kézi koordináció nem skálázódik, és megszületett a `wt-orchestrate` első verziója: plan generálás, DAG, párhuzamos dispatch. Az éles futtatások (sales-raketa projekt) hozták a harmadik fejezetet: a watchdog rendszert, a verify pipeline-t, a token budget kontrollt, és a `wt-sentinel` crash recovery-t. Márciusra érett be a spec digest pipeline, a requirement coverage tracking, a cascade failure kezelés, és a phase-end E2E tesztelés. Az első héten a rendszer 5 percig futott felügyelet nélkül. Két hónappal később 5 órát is kibír — éjszaka, alvás közben, production kódbázisokon.

FONTOS

A legfontosabb tanulság: egy orchestrációs rendszer értéke nem a ”happy path” kezelésében van — azt bárki megcsinálja. Az érték a hibakezelésben, a recovery-ben, és az eszkalációban van. A rendszer 80%-a azzal foglalkozik, ami elromolhat.

4. fejezet

A wt-tools ökoszisztéma

4.1 Áttekintés

A `wt-orchestrate` nem egyedül áll — egy tágabb eszközkészlet része, amely a teljes AI-asszisztált fejlesztési workflow-t lefedi. A `wt-tools` csomag CLI eszközöket, hookokat, memória rendszert, és projekt sablonokat tartalmaz.

4.2 A fő eszközök

4.2.1 Worktree lifecycle

Ezek az eszközök a git worktree-k teljes életciklusát kezelik — a létrehozástól a merge-ig és a cleanup-ig. Minden ágens izolált worktree-ben dolgozik, saját branch-en, anélkül, hogy a fő ágat zavarná.

Eszköz	Funkció
wt-new	Új worktree és branch létrehozása, .claude/ inicializálás
wt-list	Aktív worktree-k listázása, állapot összefoglaló
wt-status	Részletes ágens-állapot: PID, iteráció, token, modell
wt-merge	Worktree merge main-be, LLM conflict resolution
wt-close	Worktree cleanup (branch törlés, könyvtár eltávolítás)
wt-focus	Ágens-váltás: terminál fókusz a kiválasztott worktree-re
wt-add	Meglévő branch hozzáadása worktree-ként

4.2.2 Ágens futtatás

A Ralph loop a `wt-loop` által vezérelt iteratív fejlesztési ciklus. Az ágens minden iterációban kódot ír, teszteket futtat, és az eredmény alapján dönt a folytatásról. A loop automatikusan kezeli a context window limiteket, a token budgetet, és az API rate limiteket.

Eszköz	Funkció
wt-loop	Ralph iteratív loop: max-turns, token budget, model routing

Eszköz	Funkció
wt-work	Interaktív munka session worktree-ben
wt-skill-start	Skill-alapú session indítás worktree-ben

4.2.3 Orchestráció és felügyelet

Az orchestrációs réteg felelős a teljes pipeline koordinálásáért — a specifikáció feldolgozásától a végső merge-ig. A sentinel a felügyelő, amely az orchestrátort újraindítja crash esetén.

Eszköz	Funkció
wt-orchestrate	Teljes orchestrációs pipeline (ez a dokumentum tárgya)
wt-sentinel	Orchestrátor felügyelő: crash recovery, checkpoint kezelés
wt-manual	Kézi orchestrációs beavatkozás (debug, state edit)
wt-e2e-report	E2E teszt eredmények összesítése és riportálás

4.2.4 Memória és kontextus

A memória rendszer biztosítja, hogy az ágensek tanuljanak a korábbi session-ökből. Minden projekt saját memória-adatbázissal rendelkezik, amely a fontos döntéseket, tanulságokat és kontextust tárolja. A hook rendszer automatikusan injektálja a releváns memóriákat az ágens kontextusába.

Eszköz	Funkció
wt-memory	Memória CLI: remember, recall, forget, search, sync
wt-hook-memory	Automatikus memória injekció (SessionStart, PostTool)
wt-hook-memory-warmstart	Session indítás: releváns memóriák betöltése
wt-hook-memory-recall	Prompt-szintű téma-alapú memória visszahívás
wt-hook-memory-posttool	Eszköz-használat utáni kontextus kiegészítés
wt-hook-memory-save	Session végi memória mentés és extrakció

4.2.5 Projekt menedzsment

A `wt-project` rendszer biztosítja, hogy az új és meglévő projektek egyetlen paranccsal beállíthatók legyenek — hookak, parancsok, skill-ek telepítése, projekt regisztráció. Az `init` parancs idempotens: újrafuttatása frissíti a telepített fájlokat anélkül, hogy a konfigurációt felülírná.

Eszköz	Funkció
wt-project	Projekt inicializálás, hook telepítés, regisztráció
wt-config	Globális és projekt-szintű konfiguráció kezelés
wt-deploy-hooks	Claude Code hookak telepítése projekthez
wt-version	Verziószám megjelenítés
wt-usage	Token használati statisztikák

4.2.6 Csapat szinkronizáció

A team sync lehetővé teszi több fejlesztő (vagy ágens) közötti kommunikációt — üzenetküldés, aktivitás megosztás, memória szinkronizáció. A `wt-control` egy git orphan branch-et használ a kommunikációs csatornaként.

Eszköz	Funkció
wt-control	Team sync: üzenetküldés, aktivitás, koordináció
wt-control-init	Team sync inicializálás (orphan branch)
wt-control-sync	Szinkronizáció: push/pull üzenetek és aktivitás
wt-control-chat	Csevegés más ágensekkel / fejlesztőkkel
wt-control-gui	Grafikus team dashboard
wt-hook-activity	Automatikus aktivitás megosztás

4.2.7 OpenSpec és minőség

Az OpenSpec a strukturált fejlesztési workflow: a proposal-tól a design-on és a task-okon át az implementációig. Az audit és review eszközök biztosítják, hogy a minőségi kapuk ne csak az orchestrációban, hanem a kézi fejlesztésben is működjenek.

Eszköz	Funkció
wt-openspec	OpenSpec artifact kezelés (proposal, design, tasks)
wt-audit	Kód audit: security, minőség, best practices

4.3 Projekt sablonok

A `wt-project` rendszer sablon-alapú. Jelenleg két sablon érhető el, de a rendszer nyitott bármilyen technológiai stack-re:

4.3.1 wt-project-base

Az alap sablon, amely minden projekthez használható. Tartalmazza:

- Claude Code hookak (memória, activity, skill dispatch)
- `/wt:*` parancsok (orchestrate, decompose, help)
- OpenSpec skillek (fast-forward, apply, verify)
- Sentinel szabályok
- Alapértelmezett `.claude/` konfiguráció

4.3.2 wt-project-web (Next.js)

A web alkalmazásokhoz készült sablon, a `wt-project-base` kiterjesztése. Tartalmazza:

- Next.js specifikus konfigurációk
- Tesztelési stratégia (Jest + Playwright)
- Adatbázis kezelés (Prisma/Drizzle támogatás)
- `smoke_command` és `e2e_command` előkonfigurálás
- Fejlesztési szerver kezelés

4.3.3 Más irányok

A sablon rendszer szándékosan nyitott. A `wt-project-web` a Next.js irányt mutatja, de bármilyen más technológiai stack támogatható saját sablonnal:

- **wt-project-api** — REST/GraphQL API backend-ek
- **wt-project-mobile** — React Native, Flutter alkalmazások
- **wt-project-python** — Python/FastAPI/Django projektek
- **wt-project-scraper** — Adatgyűjtő és feldolgozó projektek

Ezek a sablonok a `wt-project-base`-re épülnének, és a projekt-specifikus konfigurációt, tesztelési stratégiát, és build pipeline-t adnák hozzá.

4.4 Az ökoszisztéma térképe

```
┌──────────────────────────────────────────────────────┐
│                   wt-tools ökoszisztéma                 │
├──────────────┬──────────────┬──────────────────────────┤
│  Worktree    │  Orchestráció │  Memória & Kontextus     │
│  lifecycle   │  & felügyelet │                          │
│              │              │                          │
│  wt-new      │  wt-orchestr.│  wt-memory               │
│  wt-list     │  wt-sentinel │  wt-hook-memory-*        │
│  wt-status   │  wt-manual   │                          │
│  wt-merge    │  wt-e2e-rep. │                          │
│  wt-close    │              │                          │
│  wt-loop     │              │                          │
├──────────────┼──────────────┼──────────────────────────┤
│  Projekt     │  Team Sync   │  OpenSpec & Minőség      │
│  menedzsment │              │                          │
│              │  wt-control  │  wt-openspec             │
│  wt-project  │  wt-ctrl-*   │  wt-audit               │
│  wt-config   │  wt-hook-act.│  /opsx:* skillek        │
│  wt-deploy   │              │                          │
├──────────────┴──────────────┴──────────────────────────┤
│  Projekt sablonok                                      │
│  wt-project-base │ wt-project-web │ wt-project-...     │
└──────────────────────────────────────────────────────┘
```

FONTOS

Az ökoszisztéma moduláris: nem kötelező mindent használni. A legegyszerűbb belépési pont a wt-new + wt-loop (egyetlen ágens, egyetlen worktree). Az orchestráció és a memória rendszer fokozatosan, szükség szerint kapcsolható be.

5. fejezet

Áttekintés

5.1 Mi az orchestráció?

A `wt-orchestrate` egy autonóm orchestrációs rendszer, amely egy szoftver-specifikációból kiindulva önállóan tervezi, végrehajtja és ellenőrzi a fejlesztési feladatokat. Az egész folyamat egyetlen paranccsal indul:

```
wt-orchestrate --spec docs/v3.md plan
wt-orchestrate start
```

A rendszer ezt követően:

1. **Megérti** a specifikációt (digest, requirement azonosítók)
2. **Megtervezi** a végrehajtást (change-ek, függőségi gráf)
3. **Párhuzamosan végrehajtja** a feladatokat (worktree-k, AI ágensek)
4. **Ellenőrzi** a minőséget (teszt, review, verify gate-ek)
5. **Összefésüli** az eredményt (merge, post-merge pipeline)
6. **Folytatja** a következő fázissal (auto-replan)

FONTOS

Az orchestrátor nem egyetlen nagy LLM hívás. Ez egy állapotgépes rendszer, amely 15 másodpercenként ellenőrzi az ágensek állapotát, kezeli a hibákat, és autonóm döntéseket hoz a folytatásról.

5.2 Az 5-réteg modell

Az orchestráció öt, egymásra épülő rétegből áll:

Réteg	Név	Felelősség	Főbb modulok
L1	Input	Specifikáció beolvasása, konfiguráció feloldása	config.sh, utils.sh
L2	Tervezés	Digest generálás, dekompozíció, DAG	digest.sh, planner.sh

Réteg	Név	Felelősség	Főbb modulok
L3	Végrehajtás	Worktree dispatch, Ralph loop, monitoring	dispatcher.sh, monitor.sh, watchdog.sh
L4	Minőség	Teszt, review, verify, smoke, E2E gate-ek	verifier.sh
L5	Delivery	Merge, post-merge, auto-replan	merger.sh, monitor.sh

Az L1 és L2 rétegek a "gondolkodási" fázis: a rendszer megérti a feladatot és megtervezi a megközelítést. Az L3 a "munka" réteg, ahol az AI ágensek ténylegesen kódot írnak — itt töltődik a legtöbb idő és token. Az L4 a "minőség-ellenőrzés", amely biztosítja, hogy a kész munka valóban jó legyen. Az L5 a "kézbesítés": az eredmény bekerül a fő ágba és a rendszer dönt a folytatásról.

5.3 Moduláris architektúra

A `wt-orchestrate` egyetlen belépési ponttal rendelkezik (`bin/wt-orchestrate`), de az implementáció 14 forrásmodulra van bontva a `lib/orchestration/` alatt:

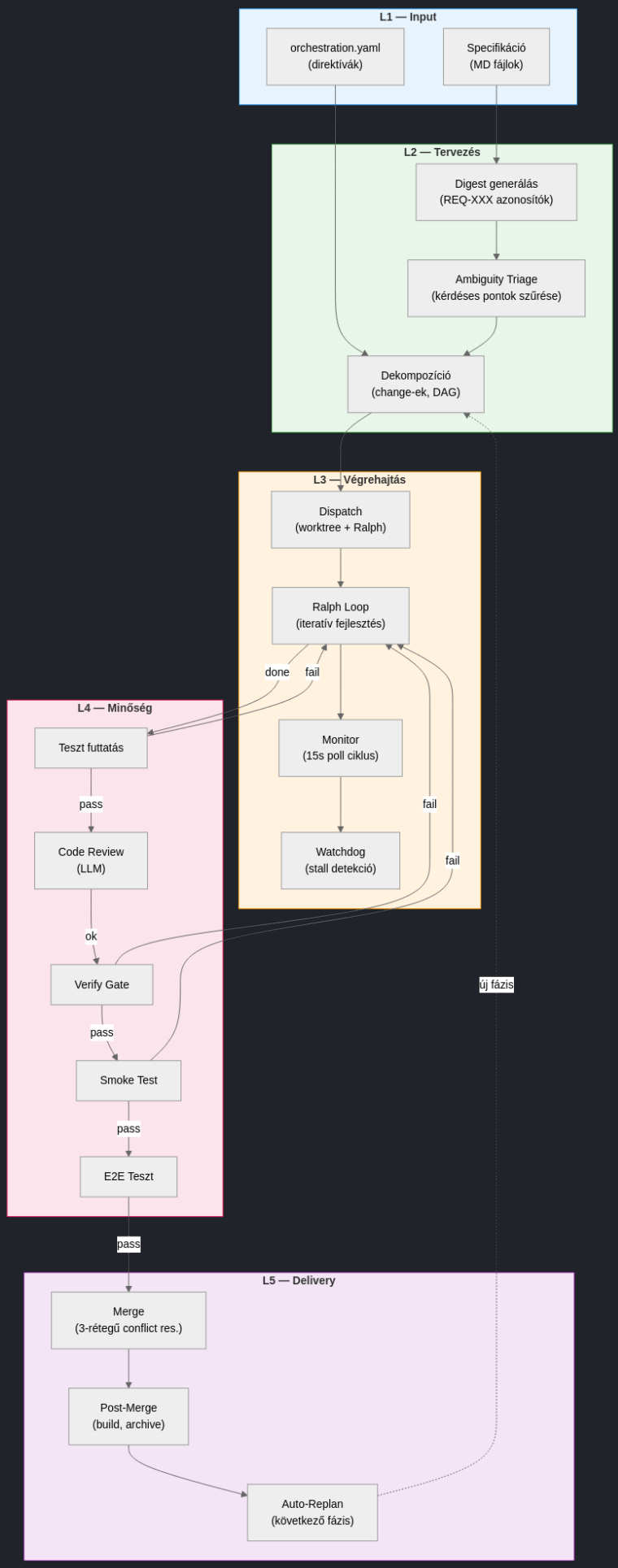
```
bin/wt-orchestrate          ← belépési pont, CLI parsing
lib/orchestration/
├── events.sh               ← JSONL eseménynapló
├── config.sh               ← konfiguráció feloldás
├── utils.sh                ← segédfüggvények
├── state.sh                ← állapotkezelés (JSON)
├── orch-memory.sh          ← memória integráció
├── watchdog.sh             ← stall detekció, eszkaláció
├── planner.sh              ← dekompozíció, validálás
├── builder.sh              ← base build health
├── dispatcher.sh           ← change lifecycle
├── verifier.sh             ← minőségi kapuk
├── merger.sh               ← merge, cleanup, archive
├── digest.sh               ← spec digest, coverage
├── reporter.sh             ← HTML riport generálás
└── monitor.sh              ← fő monitor loop
```

A modulok sorrendje számít: az `events.sh` kerül betöltésre először (minden más modul eseményeket bocsát ki), majd a `state.sh`, végül a többiek.

5.4 Állapotfájlok

Az orchestrátor három fő fájlban tárolja az állapotot:

Fájl	Tartalom
`orchestration-plan.json`	A dekompozíciós terv (change-ek, DAG, requirements)
`orchestration-state.json`	Futási állapot (change státuszok, token számok, merge queue)
`orchestration-summary.md`	Ember által olvasható összefoglaló



5.1. ábra: A teljes orchestrációs pipeline áttekintése

Ezek a fájlok a projekt gyökerében jönnek létre és nem kerülnek verziókezelésbe (`.gitignore`-ban vannak).

5.5 Egy tipikus futás

Egy közepes projekt (10-15 change, 3 párhuzamos ágens) tipikus futása:

1. **Plan** (1-2 perc): Spec feldolgozás, digest, dekompozíció
2. **Dispatch + Execution** (30-120 perc): Párhuzamos fejlesztés worktree-kben
3. **Verification** (folyamatos): Minden befejezett change-re teszt + review
4. **Merge** (folyamatos): Checkpoint policy szerint merge a fő ágba
5. **Replan** (opcionális): Ha van következő fázis, újratervezés és folytatás

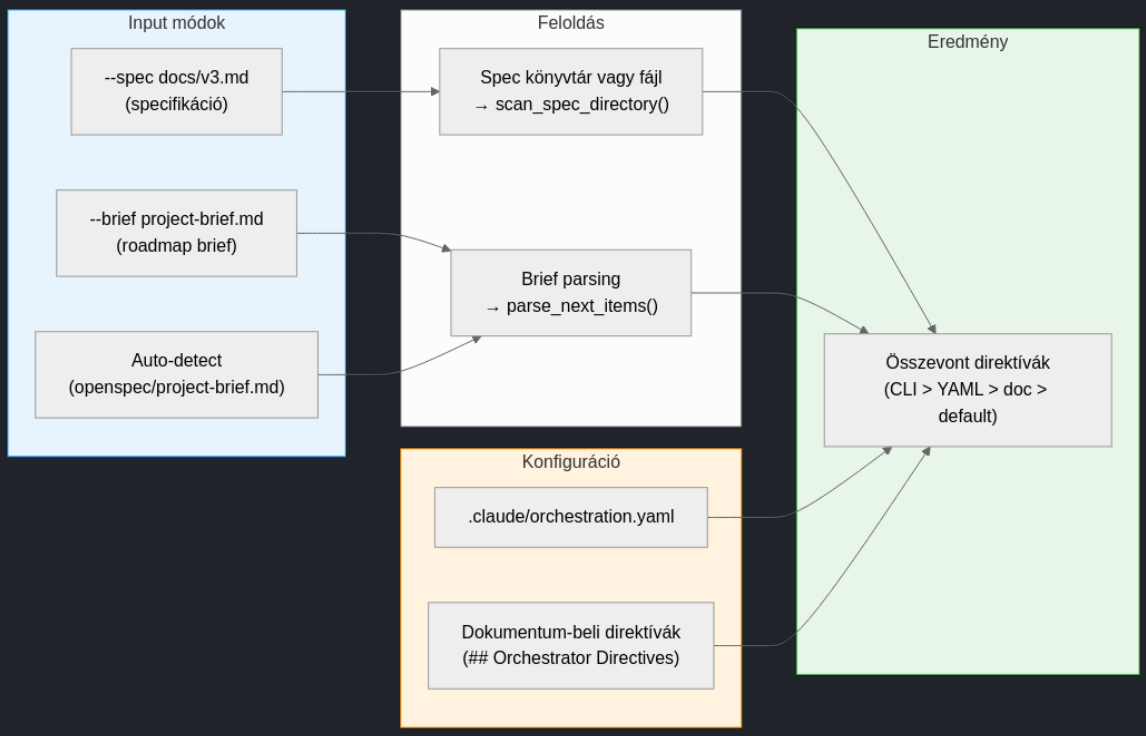
A teljes pipeline akár órákig is futhat felügyelet nélkül, miközben a watchdog és a monitor loop gondoskodik a hibakezelésről.

6. fejezet

Input és Konfiguráció

6.1 Input módok

Az orchestrátor háromféle bemenetet fogad el. Ezek kölcsönösen kizárják egymást.



6.1. ábra: Input módok és konfiguráció feloldás

6.1.1 1. Specifikáció mód (--spec)

A legtöbb projekt specifikációs dokumentumokkal dolgozik:

```
wt-orchestrate --spec docs/v3.md plan
wt-orchestrate --spec v12 plan          # rövidnév: wt/orchestration/specs/v12.md
wt-orchestrate --spec docs/ plan         # egész könyvtár
```

A `--spec` egy fájlra vagy könyvtárra mutathat. Könyvtár esetén a rendszer az összes `.md` fájlt feldolgozza, és a `scan_spec_directory()` függvény felépíti a teljes képet.

Fázis szűrés: A `--phase` opcióval egy adott fázisra lehet szűkíteni:

```
wt-orchestrate --spec docs/v3.md --phase 2 plan    # csak a 2. fázis
wt-orchestrate --spec docs/v3.md --phase "Security" plan  # szöveges szűrés
```

6.1.2 2. Brief mód (--brief)

A korábbi, egyszerűbb formátum egy `### Next` szekciót használ a roadmap elemek felsorolásához:

```
## Feature Roadmap
### Next
- Auth rendszer: JWT alapú autentikáció
- User profil: Profil szerkesztés és avatar
### Later
- Admin panel: Felhasználó kezelés
```

6.1.3 3. Auto-detect

Ha sem `--spec`, sem `--brief` nincs megadva, a rendszer automatikusan keres:

1. `openspec/project-brief.md` — ha van benne `### Next` elem
2. `openspec/project.md` — fallback
3. Hiba, ha egyik sem található

6.2 Konfiguráció

A viselkedést három szinten lehet konfigurálni. A magasabb szint felülírja az alacsonyabbat:

```
CLI flag  >  orchestration.yaml  >  dokumentum direktívák  >  alapértelmezés
```

6.2.1 orchestration.yaml

A fő konfigurációs fájl a `.claude/orchestration.yaml` (vagy `wt/orchestration.yaml`):

```yaml
# Végrehajtás
max_parallel: 3               # max párhuzamos change
merge_policy: checkpoint       # eager | checkpoint | manual
checkpoint_every: 3            # checkpoint N change után

# Tesztelés
test_command: "pnpm test"      # tesztelő parancs
test_timeout: 300              # teszt timeout (mp)
smoke_command: "pnpm build"    # smoke test parancs
smoke_blocking: true           # smoke fail blokkolja-e a merge-t

# Modellek
default_model: opus            # implementációs model
review_model: sonnet           # review model
```

```yaml
summarize_model: haiku         # összegzéshez használt model

# Review
review_before_merge: true      # LLM review a merge előtt
max_verify_retries: 2          # verify retry limit

# Automatizálás
auto_replan: true              # auto-plan a következő fázisra
context_pruning: true          # context window optimalizálás
model_routing: "off"           # model routing stratégia

# Biztonság
token_budget: 5000000          # soft limit (figyelmeztetés)
token_hard_limit: 20000000     # hard limit (emberi jóváhagyás)
time_limit: "5h"               # futási idő limit

# Hookak
post_merge_command: "pnpm db:generate"
hook_pre_dispatch: ""
hook_post_verify: ""
hook_pre_merge: ""
hook_post_merge: ""
hook_on_fail: ""
```

6.2.2 Dokumentum direktívák

A specifikáción belül is megadhatók direktívák:

```
## Orchestrator Directives
- max_parallel: 4
- merge_policy: eager
- token_budget: 100000
```

A rendszer a `parse_directives()` függvénnyel olvassa ki ezeket az értékeket.

> **FONTOS**
>
> A CLI flag mindig nyer. Ha ‘–max-parallel 5‘ van a parancssorban, az felülírja mind a YAML, mind a dokumentum direktívákat.

6.3 Direktíva referencia

Direktíva	Típus	Alapértelmezés	Leírás
max_parallel	szám	3	Max párhuzamos worktree
merge_policy	enum	checkpoint	eager/checkpoint/manual
checkpoint_every	szám	3	Checkpoint gyakoriság
test_command	string	""	Tesztelő parancs

Direktíva	Típus	Alapértelmezés	Leírás
test_timeout	szám	300	Teszt timeout (mp)
smoke_command	string	""	Smoke test parancs
smoke_blocking	bool	true	Smoke fail blokkolja-e a merge-t
smoke_fix_max_retries	szám	3	Smoke fix retry limit
e2e_command	string	""	E2E teszt parancs
e2e_mode	enum	per_change	per_change/phase_end
default_model	string	opus	Implementációs model
review_model	string	sonnet	Review model
review_before_merge	bool	false	LLM review engedélyezése
max_verify_retries	szám	2	Verify retry limit
auto_replan	bool	false	Auto-replan a fázis végén
token_budget	szám	0	Soft token limit
token_hard_limit	szám	20M	Hard token limit
time_limit	string	5h	Futási idő limit
watchdog_timeout	szám	600	Watchdog timeout (mp)
context_pruning	bool	true	Context optimalizálás
model_routing	string	off	Model routing stratégia
post_merge_command	string	""	Merge utáni parancs

7. fejezet

Digest és Triage

7.1 Mi a Digest?

A digest a specifikáció strukturált kivonatja. Egy több ezer soros spec dokumentumot gépileg feldolgozható JSON formátumra bont, amelyben minden követelmény egyedi azonosítót kap (REQ-001, REQ-002, stb.).

Ez teszi lehetővé, hogy az orchestrátor:

- **Nyomon kövesse**, melyik követelményt melyik change implementálja
- **Ellenőrizze**, hogy minden követelmény le van-e fedve
- **Felismerje** a kétértelmű pontokat, mielőtt a fejlesztés elindul

7.2 A Digest generálás lépései

7.2.1 1. Spec szkennelés (scan_spec_directory)

A rendszer feltérképezi a bemenetként kapott fájlt vagy könyvtárat:

```
wt-orchestrate --spec docs/specs/ digest
```

A szkennelés eredménye:

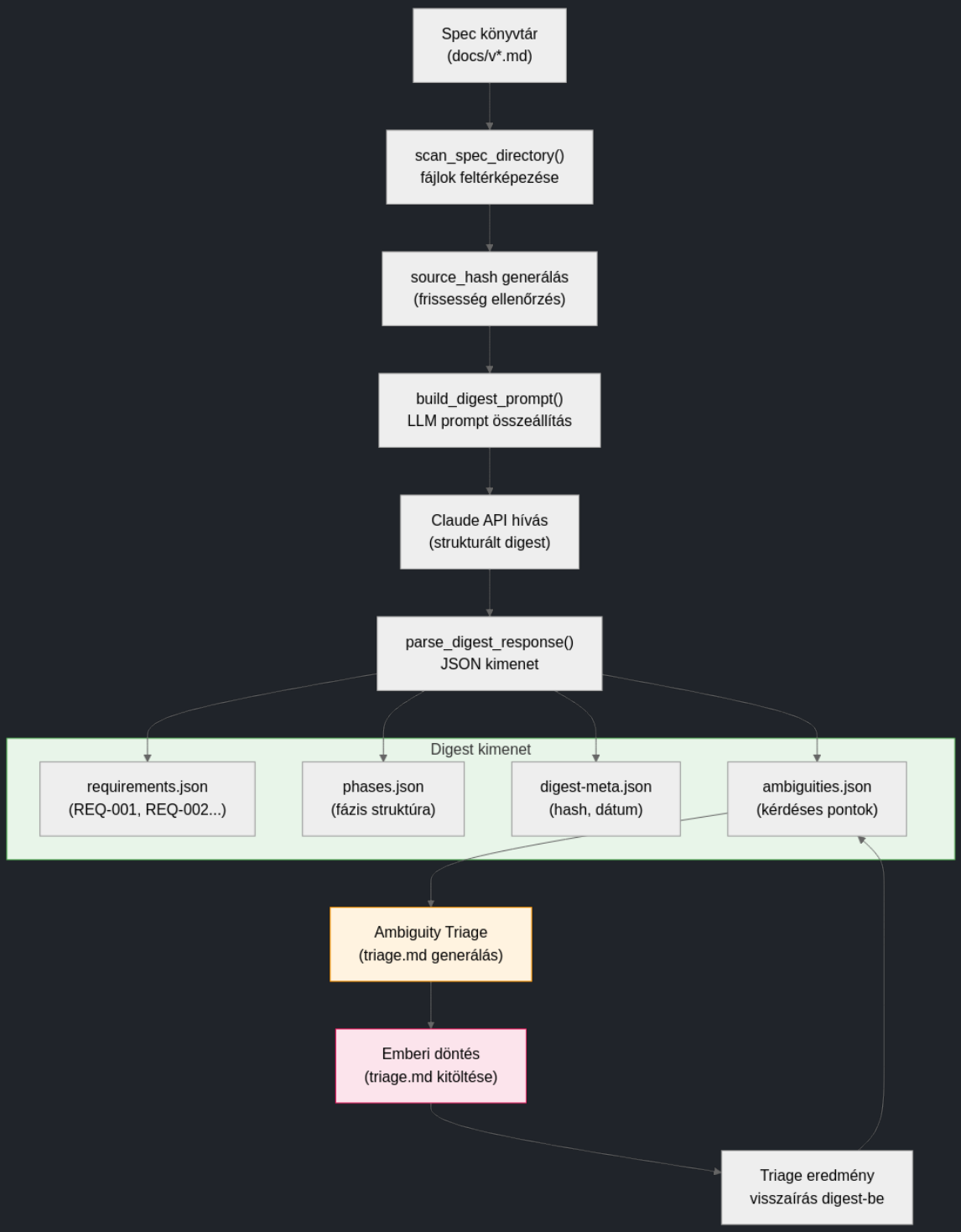
- **file_count**: hány fájl került feldolgozásra
- **source_hash**: az összes fájl tartalmából képzett hash (frissesség ellenőrzés)
- **master_file**: ha van `index.md` vagy hasonló, az a fő fájl

7.2.2 2. Prompt összeállítás (build_digest_prompt)

A rendszer egy strukturált promptot épít a Claude API-nak, amely tartalmazza az összes spec fájlt és az elvárt kimeneti formátumot.

7.2.3 3. API hívás és parsing

A Claude feldolgozza a specifikációt és strukturált JSON kimenetet ad:



7.1. ábra: A digest generálás teljes folyamata

```json
{
  "requirements": [
    {
      "id": "REQ-001",
      "title": "JWT autentikáció",
      "brief": "Token alapú auth a /api/* végpontokon",
      "priority": "high",
      "source_file": "docs/v3-security.md",
      "section": "2.1 Authentication"
    }
  ],
  "phases": [...],
  "ambiguities": [...]
}
```

7.2.4 4. ID stabilizálás (stabilize_ids)

Ha már létezik korábbi digest, a rendszer megtartja a meglévő REQ-XXX azonosítókat, hogy a követelmény-nyomkövetés ne törjön el frissítés után.

7.3 A digest kimeneti fájljai

A digest a `wt/orchestration/digest/` könyvtárba ír:

Fájl	Tartalom
requirements.json	Összes követelmény REQ-XXX azonosítóval
phases.json	Fázis struktúra (melyik fázisban mi van)
digest-meta.json	Hash, dátum, fájl számok
ambiguities.json	Kétértelmű vagy hiányos pontok

7.4 Frissesség ellenőrzés

A `source_hash` mező lehetővé teszi a gyors frissesség-ellenőrzést:

```bash
# Ha a hash nem változott, a digest naprakész
if [[ "$current_hash" == "$stored_hash" ]]; then
    echo "Digest naprakész, kihagyva"
fi
```

Ez megakadályozza a felesleges újragenerálást, ha a spec nem változott.

7.5 Ambiguity Triage

A digest generálás automatikusan azonosítja a specifikáció kétértelmű pontjait. Ezek kezelése a **triage** folyamatban történik.

7.5.1 Triage generálás

A rendszer egy `triage.md` fájlt generál, amely felsorolja a kétértelmű pontokat és döntési opciókat kínál:

```
## AMB-001: Nem egyértelmű session kezelés

**Kontextus**: A spec "session timeout" -ot említ, de nem definiálja az értéket.

**Opciók**:
- [ ] A: 30 perc (web standard)
- [ ] B: 60 perc (hosszabb munkamenet)
- [ ] C: Konfigurálható (runtime beállítás)

**Döntés**: ___
```

7.5.2 Emberi döntés

A triage fájlt egy ember (vagy a sentinel) kitölti. A döntés visszaíródik az `ambiguities.json`-be, és a planner figyelembe veszi a dekompozíciónál.

7.5.3 Automatikus triage mergelés

A `merge_triage_to_ambiguities()` és `merge_planner_resolutions()` függvények biztosítják, hogy:

- Az emberi döntések bekerüljenek a digest-be
- A planner által hozott döntések szintén megmaradjanak
- Ellentmondás esetén az emberi döntés nyer

FONTOS

A triage gate az egyetlen pont, ahol az orchestrátor megáll és emberi beavatkozást kér. Minden más döntést autonóm módon hoz. Ha nincs kétértelmű pont, a triage lépés automatikusan kimarad.

7.6 Requirement Coverage

A digest lehetővé teszi a követelmény-lefedettség nyomon követését az egész pipeline-on keresztül:

```
wt-orchestrate coverage
```

Kimenet:

```
Requirement Coverage: 12/15 (80%)
  ✓ REQ-001: JWT autentikáció          → change/auth-system (merged)
  ✓ REQ-002: User profil               → change/user-profile (running)
  ✗ REQ-003: Admin panel                → (not assigned)
  ...
```

A `update_coverage_status()` függvény automatikusan frissíti a lefedettséget, amikor egy change státusza változik (merged, failed, stb.).

8. fejezet

OpenSpec — A Spec-Driven Workflow

8.1 Mi az OpenSpec?

Az OpenSpec egy strukturált fejlesztési workflow, amely minden változtatást (change) fázisokon vezet végig, mielőtt az implementáció elkezdődne. A cél: az AI ágens ne "csak kódoljon", hanem *értse meg* a feladatot, és nyomon követhető döntéseket hozzon.

A fázisok:

```
Proposal (MIÉRT) → Design (HOGYAN) → Specs (MIT) → Tasks (CSINÁLD) → Implementáció
```

8.2 Az artifact-ok

8.2.1 Proposal

A *miért* dokumentum: motiváció, scope, sikerkritériumok. Az orchestrátor a plan-ből **előre generálja** a proposal-t a worktree-ben, a roadmap item és scope alapján — az ágens ezt kapja kiindulásként.

8.2.2 Design

A *hogyan* dokumentum: technikai döntések ADR (Architecture Decision Record) formátumban, kockázatok, alternatívák. Az ágens generálja a proposal-ból.

8.2.3 Specs

A *mit* dokumentum: részletes követelmények BDD (Given/When/Then) formátumban, delta műveletek (ADDED, MODIFIED, REMOVED). Minden követelmény tesztelhető szcenáriókkal.

8.2.4 Tasks

A *csináld* lista: checkbox-os feladatlista, amit az ágens az implementáció során egyenként pipál. A `[?]` jelölés manuális (emberi) feladatot jelöl — API kulcs beszerzés, külső szolgáltatás beállítás.

```
- [x] Add JWT middleware to /api/* routes
- [x] Create login endpoint with bcrypt
- [ ] Add refresh token rotation
- [?] Configure OAuth provider credentials
```

8.3 Integráció az orchestrációval

8.3.1 Dispatch fázis

A `dispatcher.sh` a worktree létrehozásakor:

1. Létrehozza a change könyvtárat (`openspec new change`)
2. **Előre generálja a proposal-t** a plan scope-jából
3. Bemásolja a spec kontextust (digest módban)
4. Hozzáfűzi a cross-cutting fájlok listáját, a szomszédos change-ek státuszát, és a memória kontextust

8.3.2 Ralph loop integráció

A Ralph loop az OpenSpec artifact-ok állapota alapján dönt:

```
Ha nincs tasks.md → action: ff (fast-forward, minden artifact létrehozása)
Ha van tasks.md  → action: apply (implementáció)
Ha minden kész   → action: done
```

Az `ff` fázisban az ágens a `/opsx:ff` skillt futtatja: egyetlen session-ben létrehozza a design-t, specs-et és tasks-ot a proposal-ból. Az `apply` fázisban a `/opsx:apply` skillt futtatja: a tasks.md alapján implementálja a feladatokat.

8.3.3 FF→Apply láncolás

Korábban az artifact-generálás (ff) és az implementáció (apply) két külön iterációban futott, és az iteráció-határ kontextusvesztést okozott. A jelenlegi verzió egyetlen iteráción belül láncolja: az ff befejezése után azonnal indul az apply, friss kontextussal.

8.4 Interaktív vs. orchestrált használat

Mód	Parancsok	Mikor
Interaktív	/opsx:new → /opsx:continue → /opsx:apply → /opsx:verify → /opsx:archive	Fejlesztő egyedül dolgozik, lépésenként
Fast-forward	/opsx:ff → /opsx:apply → /opsx:archive	Fejlesztő, de nincs idő lépésenkéntre
Orchestrált	Automatikus --- a dispatcher és Ralph loop kezeli	wt-orchestrate start

8.5 Nyomkövethetőség

Az OpenSpec teljes nyomkövetési láncot biztosít:

```
Spec szekció (v3.md)
  → Requirement ID (REQ-042)
    → Change hozzárendelés (platform-dashboard)
      → Proposal: "Ez a change a REQ-042-t fedi le"
        → Tasks: "KPI kártyák hozzáadása"
          → Git commit: a REQ-042-t implementáló kód
```

A verify gate nem csak a kód minőségét, hanem a **követelmény-megfelelőséget** is ellenőrzi: a review prompt tartalmazza a hozzárendelt REQ-XXX azonosítókat.

FONTOS

Az OpenSpec az orchestráció és a végrehajtás közötti szerződés. Az orchestrátor létrehozza a proposal-t; az ágens létrehozza a design-t, specs-et és tasks-ot; a verify gate ellenőrzi az implementációt. Ha bármelyik fázis hiányzik, a rendszer tudja, hol tart és mi a következő lépés.

8.6 Limitációk

- **FF kudarcok**: Az `/opsx:ff` néha nem hozza létre a tasks.md-t (nagy spec, truncált LLM válasz). Ilyenkor a loop fallback tasks-ot generál a proposal-ból
- **Manuális feladatok**: A `[?]` jelölésű feladatok (API kulcsok, external setup) emberi beavatkozást igényelnek — az orchestráció ezeket nem tudja automatizálni
- **Nagy change-ek**: 14+ requirement egy change-ben → megbízhatatlan. A planner max 6 REQ/change szabályt alkalmaz

9. fejezet

Tervezés

9.1 A dekompozíció célja

A tervezési fázis a specifikációt (vagy digest-et) **végrehajtható change-ekre** bontja. Minden change egy önálló, jól definiált fejlesztési feladat, amelyet egy AI ágens egy worktree-ben hajt végre.

9.2 Tervezési módszerek

Két tervezési módszer létezik, a `plan_method` direktíva vezérli:

9.2.1 API mód (alapértelmezett)

Egyetlen Claude API hívás végzi a dekompozíciót:

```
wt-orchestrate --spec docs/v3.md plan   # plan_method: api (default)
```

Előnyök:

- Gyors (1-2 perc)
- Kevés token használat
- Determinisztikus kimenet

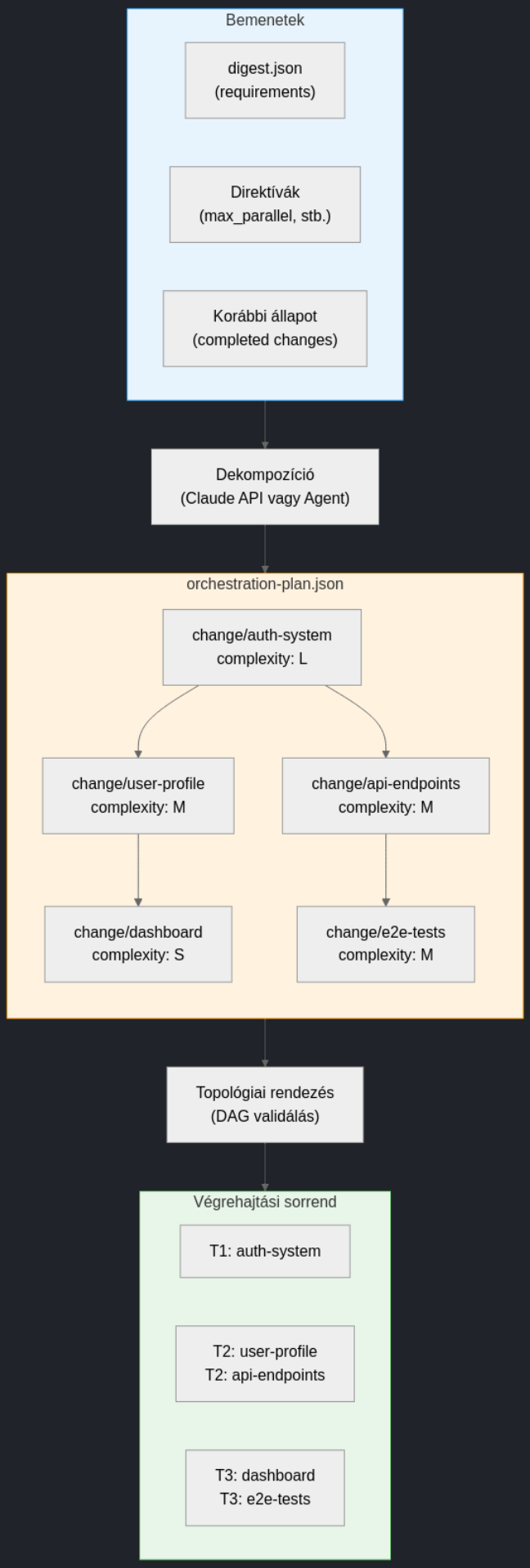
9.2.2 Agent mód

Egy dedikált worktree-ben egy Ralph loop végzi a tervezést, több iterációban:

```yaml
# orchestration.yaml
plan_method: agent
plan_token_budget: 500000
```

Ez a módszer akkor hasznos, ha:

- A specifikáció nagyon összetett
- A planner-nek kódot kell olvasnia a jó dekompozícióhoz
- A tesztelési infrastruktúra felderítése szükséges



9.1. ábra: Dekompozíció és DAG generálás

9.3 A Plan struktúra

A `cmd_plan()` eredménye az `orchestration-plan.json`:

```
{
  "plan_version": 1,
  "brief_hash": "a1b2c3d4",
  "plan_phase": "phase-2",
  "plan_method": "api",
  "changes": [
    {
      "name": "auth-system",
      "scope": "JWT autentikáció implementálása a /api/* végpontokon",
      "complexity": "L",
      "change_type": "feature",
      "depends_on": [],
      "roadmap_item": "Auth rendszer",
      "requirements": ["REQ-001", "REQ-004"],
      "also_affects_reqs": ["REQ-012"],
      "model": null,
      "skip_review": false,
      "skip_test": false
    },
    {
      "name": "user-profile",
      "scope": "Profil szerkesztés és avatar feltöltés",
      "complexity": "M",
      "depends_on": ["auth-system"],
      "requirements": ["REQ-002", "REQ-003"]
    }
  ]
}
```

9.3.1 Mezők magyarázata

Mező	Leírás
name	Egyedi azonosító (ez lesz a branch neve: change/auth-system)
scope	Részletes leírás az ágens számára
complexity	S (small), M (medium), L (large) --- token limit meghatározáshoz
change_type	feature, fix, refactor, test, docs, config
depends_on	Függőségek (más change nevek)
requirements	Hozzárendelt REQ-XXX azonosítók
also_affects_reqs	Cross-cutting követelmények (figyelemfelhívás)
model	Change-specifikus model override (null = default)
skip_review	Review gate kihagyása (pl. config change-eknél)
skip_test	Test gate kihagyása (pl. docs change-eknél)

9.4 Topológiai rendezés (DAG)

A `topological_sort()` függvény a `depends_on` mezők alapján egy irányított aciklikus gráfot (DAG) épít, és topológiai sorrendben adja vissza a change-eket.

Szabályok:

1. Függőség nélküli change-ek párhuzamosan indulhatnak
2. Egy change csak akkor indulhat, ha minden függősége `merged` státuszban van
3. Körkörös függőség → hiba a plan generálásnál

Példa: Ha A ← B és A ← C, de D ← B,C:

```
T1: A (nincs függőség)
T2: B, C (párhuzamosan, A merged után)
T3: D (B és C merged után)
```

9.4.1 Körkörös függőség detekció

```
# A self-test tartalmaz tesztet:
wt-orchestrate self-test
# → PASS: circular dependency detected
```

Ha a planner körkörös függőséget generál, a rendszer hibát dob és a plan-t el kell vetni.

9.5 Tesztelési infrastruktúra felderítés

A `detect_test_infra()` függvény a plan fázisban felméri a projekt tesztelési képességeit:

- **Framework**: vitest, jest, pytest, mocha
- **Config fájlok**: vitest.config.ts, jest.config.js, stb.
- **Teszt fájlok**: `*.test.*`, `*.spec.*`, `test_*.py`
- **Helper könyvtárak**: `src/test/`, `__tests__/`

Ez az információ bekerül a planner prompt-ba, hogy a change-ek scope-ja tartalmazza a tesztelést is.

9.6 Spec összegzés nagy dokumentumokhoz

Ha a spec túl nagy (sok ezer token), a `summarize_spec()` egy haiku modellel összegzi az anyagot, mielőtt a planner megkapná:

1. Az összegző kivonja a fázis struktúrát
2. Azonosítja a kész és hiányzó részeket
3. A planner csak a releváns fázist kapja meg

FONTOS

A plan mindig megtekinthető a generálás után: ‘wt-orchestrate plan –show‘. A ‘start‘ parancs kiadása előtt érdemes ellenőrizni, hogy a dekompozíció ésszerű-e.

9.7 Plan validálás

A generált plan-t a rendszer automatikusan validálja:

- Minden change névnek egyedinek kell lennie
- A `depends_on` hivatkozásoknak létező change-ekre kell mutatniuk
- Körkörös függőség nem megengedett
- A `complexity` értéknek S, M vagy L-nek kell lennie

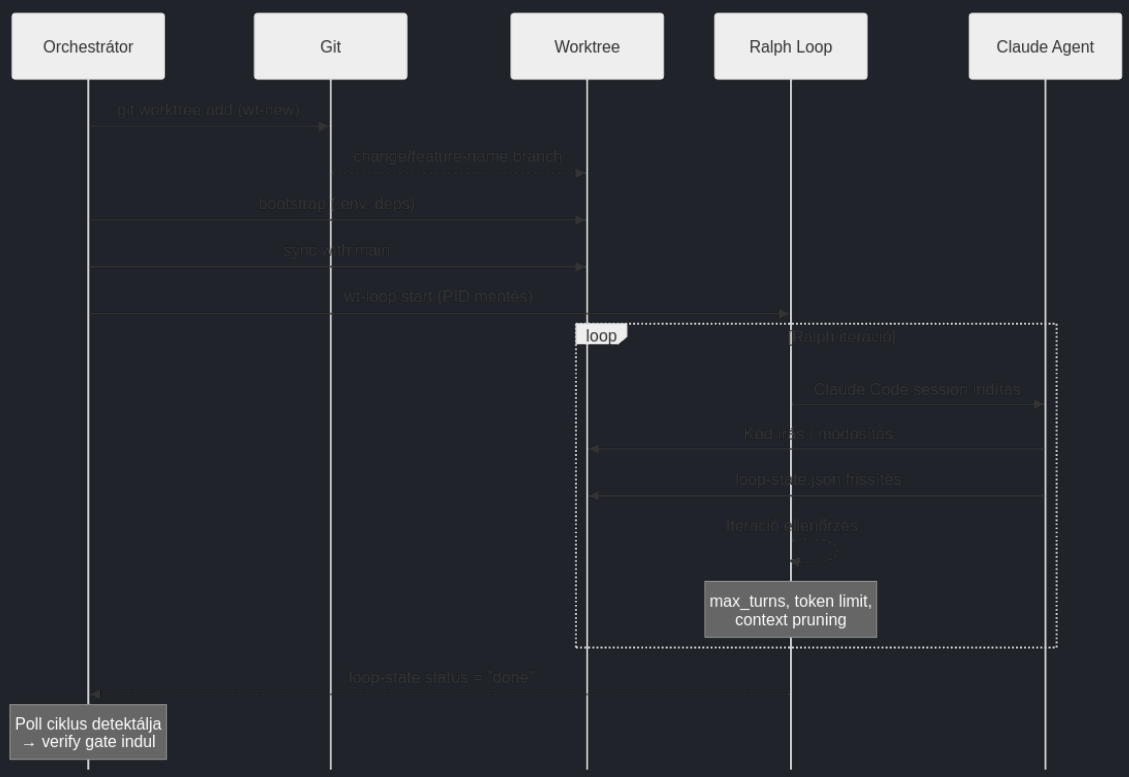
Ha a validálás sikertelen, a plan nem mentődik és hibaüzenetet kap a felhasználó.

10. fejezet

Végrehajtás

10.1 Dispatch: a change elindul

Amikor egy change minden függősége teljesült (merged státusz), a `dispatch_ready_changes()` elindítja a végrehajtást. A folyamat:



10.1. ábra: A dispatch és Ralph loop teljes folyamata

10.1.1 1. Worktree létrehozás

```
wt-new <change-name>
# → .claude/worktrees/<change-name>/
# → branch: change/<change-name>
```

A `wt-new` egy izolált git worktree-t hoz létre, ahol az ágens szabadon dolgozhat anélkül, hogy a fő ágat érintené.

10.1.2 2. Bootstrap

A `bootstrap_worktree()` gondoskodik arról, hogy a worktree működőképes legyen:

- `.env` fájlok másolása a projekt gyökeréből
- Függőségek telepítése (ha `package-lock.json` újabb)
- Az `.env.local`, `.env.development` fájlok szintén átkerülnek

10.1.3 3. Szinkronizálás a fő ággal

A `sync_worktree_with_main()` biztosítja, hogy a worktree a legfrissebb main-re épüljön:

- Ha a worktree up-to-date → kihagyás
- Ha van eltérés → `git merge main` a worktree branch-be
- Generált fájlok (lock file, `.tsbuildinfo`) konfliktusa → automatikus `--ours` feloldás
- Valódi kód konfliktus → merge abort, figyelmeztetés

10.1.4 4. Ralph Loop indítás

```
wt-loop start --change <name> --model <model_id> --max-turns <N>
```

A Ralph PID-je mentésre kerül az állapotfájlba.

10.2 A Ralph Loop

A Ralph loop a `wt-loop` paranccsal indul és egy iteratív fejlesztési ciklust hajt végre:

10.2.1 Iterációs ciklus

Minden iterációban:

1. **Claude Code session** indul a worktree-ben
2. Az ágens megkapja a **scope**-ot (feladat leírás) és az opcionális **retry context**-et
3. Az ágens **kódot ír**, teszteket futtat, fájlokat módosít
4. A `loop-state.json` frissül az iteráció végén

10.2.2 loop-state.json

Az iteráció állapota egy JSON fájlban:

```
{
  "status": "running",           // running | done | error
  "iteration": 5,
  "max_turns": 20,
  "model": "claude-opus-4-6",
  "started_at": "2026-03-10T14:00:00Z",
  "tokens_used": 450000,
```

```
  "input_tokens": 300000,
  "output_tokens": 150000,
  "cache_read_tokens": 120000,
  "last_activity": "2026-03-10T14:15:30Z"
}
```

10.2.3 Befejezési feltételek

A Ralph loop az alábbi esetekben áll le:

Feltétel	Eredmény
Az ágens jelzi: kész	status: "done"
max_turns elérve	status: "done" (kényszerített)
Token limit elérve	status: "done"
Fatális hiba	status: "error"

10.2.4 Context Pruning

A `context_pruning: true` direktíva (alapértelmezett) lehetővé teszi a context window optimalizálást:

- Az ágens korábbi iterációinak kimenete összegződik
- A teljes konverzáció helyett csak a lényeges rész marad
- Ez csökkenti a token használatot és javítja a minőséget

10.3 Model Routing

A `model_routing` direktíva lehetővé teszi, hogy különböző change-ek különböző modelleket használjanak:

- **`off`** (alapértelmezett): minden change a `default_model`-t használja
- Change-specifikus override a plan-ben: `"model": "sonnet"`

Tipikus konfiguráció:

```
default_model: opus        # bonyolult feature-ök
review_model: sonnet       # code review (gyorsabb, olcsóbb)
summarize_model: haiku     # összegzés (leggyorsabb)
```

10.3.1 Complexity alapú routing

A planner a `complexity` mezővel jelzi a change méretét:

Complexity	Jellemző	Tipikus model
S (Small)	Config, docs, egyszerű fix	sonnet
M (Medium)	Feature, refactor	opus
L (Large)	Architektúra, cross-cutting	opus

10.4 Párhuzamos végrehajtás

A `max_parallel` direktíva szabályozza, hogy egyszerre hány worktree-ben dolgozhat ágens:

```
max_parallel: 3

T=0    [auth-system] ──────────────────────────── done → verify
T=0    [api-endpoints] ────────────────────── done → verify
T=0    [config-update] ─── done → verify → merge
T=15   [user-profile] ──────────────────── (dispatch after auth merged)
T=30   [dashboard] ──────────── (dispatch after slot free)
```

A dispatch logika:

1. Számold a `running` + `dispatched` + `verifying` change-eket
2. Ha kevesebb, mint `max_parallel` → keresd a következő kész change-et
3. Egy change "kész" ha: `pending` státuszú ÉS minden `depends_on` entry `merged`

> **FONTOS**
>
> A párhuzamos végrehajtás nem jelenti, hogy az összes change egyszerre fut. A DAG és a max_parallel együtt határozza meg a végrehajtási ütemezést. Egy 15 change-es plan 3 párhuzamossággal tipikusan 5-6 "hullámban" fut le.

10.5 Pause és Resume

A futás bármikor felfüggeszthető és folytatható:

```
wt-orchestrate pause auth-system    # egy change
wt-orchestrate pause --all          # minden change
wt-orchestrate resume auth-system   # folytatás
wt-orchestrate resume --all         # minden folytatása
```

A `pause` megállítja a Ralph loop-ot (PID kill), de a worktree megmarad. A `resume` újraindítja a loop-ot, opcionálisan retry context-tel.

10.6 Token tracking

Minden change token használata valós időben nyomon követhető:

- `tokens_used`: összes token (input + output)
- `input_tokens`, `output_tokens`: részletes bontás
- `cache_read_tokens`, `cache_create_tokens`: prompt cache statisztikák

A `_prev` mezők a korábbi iterációk összegét tárolják (restart után is pontos marad a számolás).

11. fejezet

A Sentinel

11.1 Miért kell felügyelő a felügyelőnek?

Az orchestrátor maga is egy process — és a processek néha meghalnak. OOM kill, API timeout, váratlan exception, broken pipe. Ha az orchestrátor éjszaka, felügyelet nélkül fut és leáll, reggel egy félkész állapotot talál a fejlesztő.

A `wt-sentinel` erre a problémára ad választ: egy supervisor, amely figyeli az orchestrátort, és ha az leáll, megpróbálja újraindítani. Két üzemmódban működik: **bash mód** (költségmentes, determinisztikus) és **ágens mód** (LLM-alapú, intelligens döntéshozatal).

11.2 Bash Sentinel

A bash sentinel egy önálló script, amely a `wt-orchestrate start` parancsot csomagolja:

```
wt-sentinel --spec docs/v3.md --time-limit 5h
# ↑ minden opció az orchestrátornak adódik tovább
```

11.2.1 Crash Recovery exponenciális backoff-fal

Ha az orchestrátor leáll, a sentinel nem azonnal indítja újra — exponenciális backoff-fal vár:

```
Crash 1: vár 30s  (+ 0-25% jitter)
Crash 2: vár 60s
Crash 3: vár 120s
Crash 4: vár 240s (maximum)
Crash 5: feladja → SENTINEL_FAILED event
```

Ha az orchestrátor legalább 5 percig futott (sustained run), a számláló és a backoff alaphelyzetbe áll. Ez megkülönbözteti az indítási hibákat (konfigurációs probléma, ami mindig azonnal megöli) a futásidejű crashektől (API timeout, ami egyszer fordul elő és utána rendben megy).

11.2.2 Élőség-detekció

A sentinel nem a logot elemzi — az events fájl módosítási idejét (mtime) figyeli:

```
Orchestrátor → WATCHDOG_HEARTBEAT (15s-enként) → events.jsonl mtime frissül
Sentinel → pollolja az mtime-ot (10s-enként)
  → Ha >180s óta nem változott → az orchestrátor beragadt
  → SIGTERM → 30s várakozás → ha kell, SIGKILL → újraindítás
```

A `WATCHDOG_HEARTBEAT` eseményt a monitor loop minden ciklusban kibocsátja. Ha ez elmarad 3 percnél tovább, a sentinel beavatkozik.

11.2.3 Állapot-helyreállítás

Minden újraindítás előtt a sentinel megpróbálja javítani az állapotot:

1. **Event-alapú rekonstrukció**: Ha az `orchestration-state.json` régebbi, mint az events log, a sentinel az események visszajátszásából rekonstruálja az állapotot — melyik change hol tartott, mennyi tokent használt
2. **Halott PID detekció**: A `running` státuszú change-ek PID-jét ellenőrzi. Ha a process már nem él, a change `stalled` státuszba kerül (a watchdog fog foglalkozni vele)
3. **Állapot normalizálás**: Ha az orchestrátor státusza `running` de nincs process, `stopped`-ra állítja

11.2.4 Kilépési osztályozás

A sentinel különbséget tesz **végleges** és **átmeneti** kilépés között:

Állapot	Kategória	Sentinel művelete
done	Végleges	Leáll, nem indít újra
stopped	Végleges	Leáll
time_limit	Végleges	Leáll
plan_review	Végleges	Leáll (emberi döntés kell)
Bármilyen crash	Átmeneti	Újraindít backoff-fal

11.3 Ágens Sentinel

A `/wt:sentinel` skill egy Claude Code session-ként fut, és intelligens döntéseket hoz:

11.3.1 Tiered beavatkozás

Az ágens sentinel a következő elv alapján működik: **ne avatkozz bele az orchestráció szintű problémákba** — az orchestrátornak van beépített recovery-je.

Tier	Mikor	Példák
Tier 1 --- Nem avatkozik	Orchestráció szintű probléma	merge-blocked, test fail, verify retry, replan
Tier 2 --- Beavatkozik	Process szintű probléma	Crash, hang, terminális állapot, checkpoint

Merge conflict? Az orchestrátor 3-rétegű conflict resolution-nel kezeli. Test failure? Az orchestrátor retry-olja. Egyedi change elbukott? Az orchestrátor folytatja a többivel. Az ágens sentinel csak akkor lép közbe, amikor az orchestrátor processze maga akad el.

11.3.2 Checkpoint kezelés

- **Periodikus checkpoint** (N merge után): automatikus jóváhagyás
- **Budget checkpoint** (token hard limit): escalate a felhasználóhoz
- **Egyéb checkpoint**: felhasználói döntés kell

11.3.3 Autonómia szabályok

A sentinel speciális szabálykészlettel dolgozik:

- **Soha ne kérdezz mielőtt javítanál** — ha bug van, javítsd, commitold, indítsd újra
- **Soha ne kérdezz mielőtt újraindítanál** — crash után cleanup és restart, megerősítés nélkül
- **A polling soha ne álljon le magától** — fix után folytasd, restart után folytasd, context compact után folytasd

FONTOS

A sentinel kettős védelmet biztosít: a bash sentinel költségmentesen figyeli a process-t és automatikusan újraindít, míg az ágens sentinel LLM-mel analizálja a helyzetet és intelligens döntéseket hoz. Éles futtatásoknál a bash sentinel az alap; az ágens sentinel opcionális, E2E teszteléshez és fejlesztéshez hasznos.

11.4 Sentinel események

A sentinel közvetlenül ír az events JSONL-be (nem függ az `events.sh` modultól):

Esemény	Jelentés
SENTINEL_RESTART	Orchestrátor leállt, újraindítás (exit code, backoff, crash count)
SENTINEL_FAILED	5 rapid crash, a sentinel feladta
STATE_RECONSTRUCTED	Állapot rekonstruálva events-ből

12. fejezet

Monitor és Watchdog

12.1 A Monitor Loop

Az orchestrátor szíve a `monitor_loop()` függvény, amely a `start` parancs kiadása után végtelenül fut. Feladata: 15 másodpercenként ellenőrizni az összes aktív change állapotát, és a megfelelő műveletet végrehajtani.

12.1.1 A poll ciklus

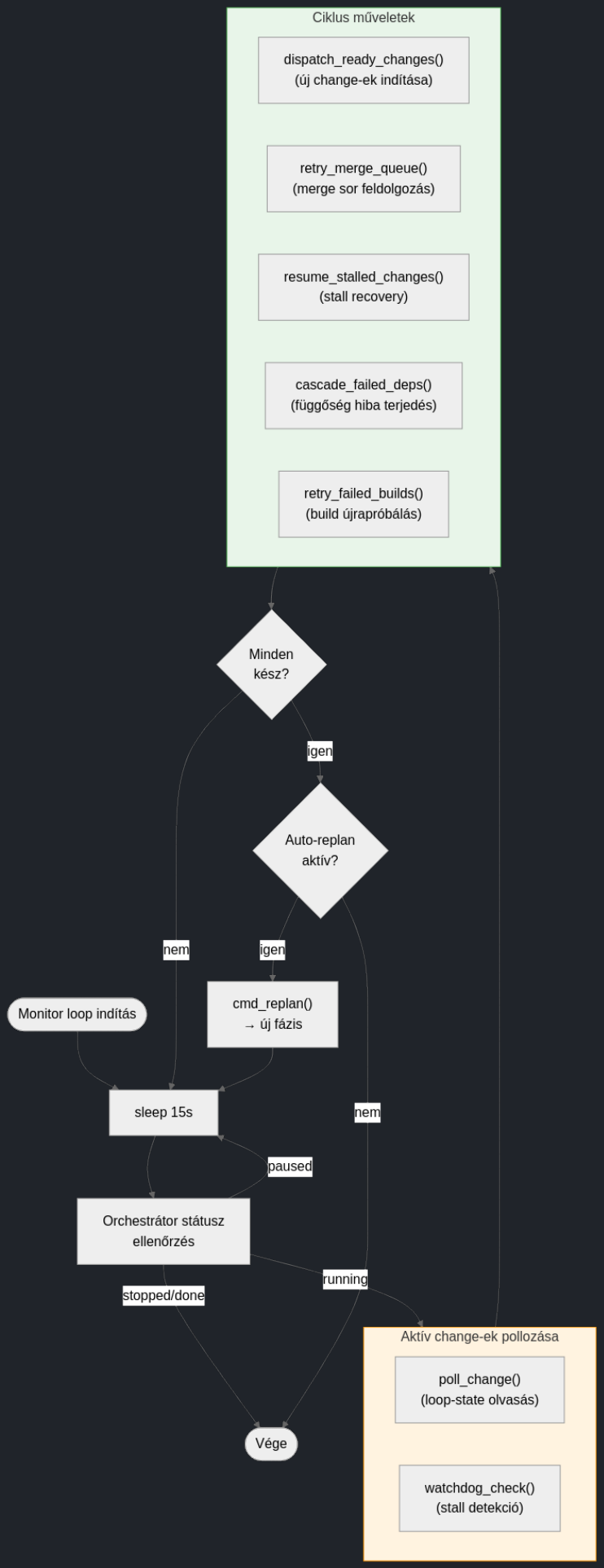
Minden 15 másodperces ciklusban a következők történnek:

1. **Állapot ellenőrzés**: Az orchestrátor státuszának (running/paused/stopped) vizsgálata
2. **Aktív change-ek pollozása**: `poll_change()` hívás minden `running` és `verifying` change-re
3. **Watchdog ellenőrzés**: `watchdog_check()` hívás stall detekció céljából
4. **Felfüggesztett change-ek**: Ellenőrzés, hogy a `paused` vagy `budget_exceeded` change-ek befejeztek-e
5. **Token budget**: Soft és hard limit ellenőrzés
6. **Verify-failed recovery**: Újrapróbálás, ha a retry limit engedi
7. **Dispatch**: Új change-ek indítása, ha van szabad slot
8. **Merge queue**: Merge sor feldolgozása
9. **Stall recovery**: Elakadt change-ek újraindítása
10. **Cascade failure**: Sikertelen függőségek propagálása
11. **Replan**: Ha minden kész és `auto_replan` aktív, következő fázis

12.1.2 poll_change()

A `poll_change()` a legfontosabb függvény. Egy change esetén:

- Olvassa a `loop-state.json` fájlt a worktree-ből
- Frissíti a token számlálókat az állapotfájlban
- Ha `status == "done"` → elindítja a verify pipeline-t (teszt → review → verify → smoke → E2E)
- Ha `status == "error"` → retry vagy fail



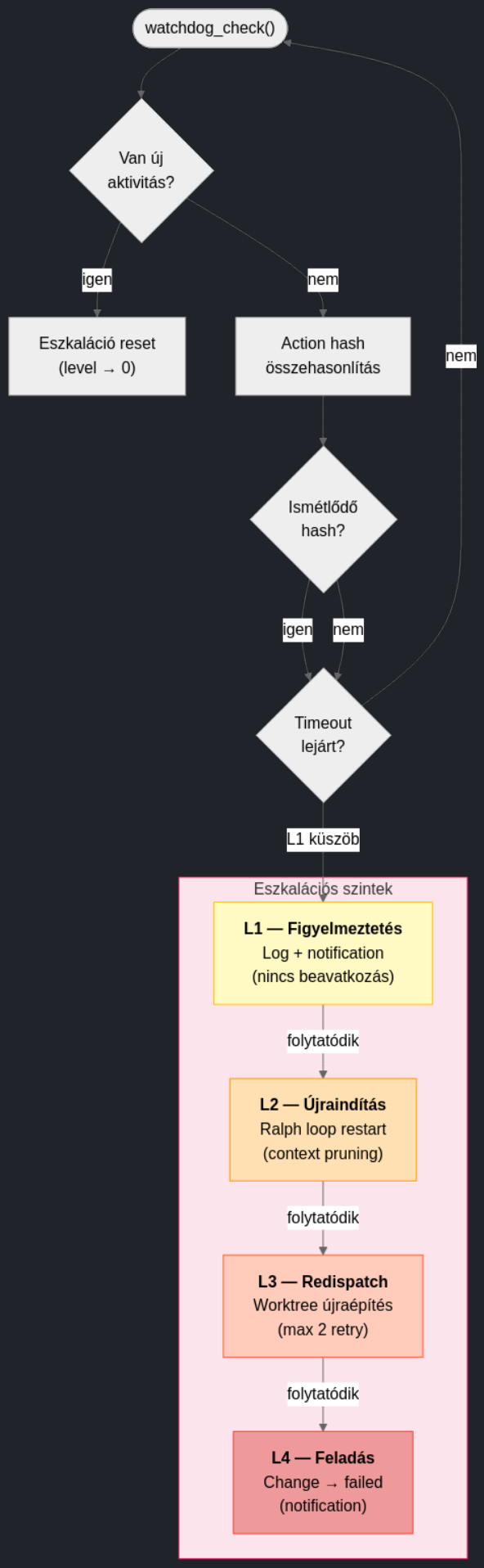
12.1. ábra: A monitor loop ciklusa

12.1.3 Active time tracking

A monitor nem a faliórát méri, hanem az **aktív időt**:

- Csak akkor számol, ha legalább egy Ralph loop fut (`any_loop_active()`)
- Token budget várakozás alatt nem számlál
- Pause alatt nem számlál
- Az aktív idő restart-ok között is kumulálódik

```
Fali idő:    |▓▓▓
```



12.2. ábra: A watchdog eszkalációs szintjei

12.2.2 Eszkalációs szintek

Szint	Művelet	Leírás
L1	Figyelmeztetés	Log bejegyzés + notification. Nincs beavatkozás.
L2	Újraindítás	A Ralph loop leállítása és újraindítása. Context pruning aktiválódik.
L3	Redispatch	A teljes worktree újraépítése. Max max_redispatch (alapértelmezés: 2) alkalommal.
L4	Feladás	A change failed státuszba kerül. Notification küldése.

Az eszkalációs szintek fokozatos beavatkozást jelentenek. Az L1 csak figyel — hátha az ágens magától megoldja a problémát (pl. hosszú build futás). Az L2 újraindítja a loop-ot friss context-tel — ez a leggyakoribb helyreállítás, kb. 70%-ban sikeres. Az L3 mindent elölről kezd: friss worktree, friss branch, teljes újradispatch — drága, de ha az L2 nem segít, ez az utolsó esély. Az L4 feladja a harcot: jobb egy change-et elveszíteni, mint végtelen tokeneket égetni.

12.2.3 Recovery detekció

Ha egy eszkalált change újra aktivitást mutat (pl. L2 újraindítás után a hash megváltozik), az eszkaláció **automatikusan resetelődik**:

```
L2 (restart) → aktivitás detektálva → level reset → L0
```

FONTOS

A watchdog az egyetlen biztonsági háló, amely megakadályozza, hogy egy elakadt ágens végtelen ideig fogyassza a tokeneket. Éles futtatásnál a watchdog_timeout és max_redispatch értékeket a projekt jellegéhez kell igazítani.

12.3 Token biztonsági hálók

A monitor loop két szintű token védelmet biztosít:

12.3.1 Soft limit (token_budget)

Ha az összes change token használata meghaladja a `token_budget` értéket:

- A futó loop-ok befejezhetik az aktuális iterációt
- Új change-ek **nem** indulnak
- Amint a budget alá kerül a használat, a dispatch újraindul

12.3.2 Hard limit (token_hard_limit)

Ha az összes token (beleértve a korábbi replan ciklusokat is) eléri a hard limitet:

- **Checkpoint** aktiválódik: az orchestrátor megáll
- Emberi jóváhagyás szükséges a folytatáshoz
- Jóváhagyás után a limit feljebb tolódik

```
wt-orchestrate approve   # jóváhagyás, folytatás
```

12.4 Időlimit

A `time_limit` direktíva (alapértelmezés: 5 óra) az **aktív** futási időt korlátozza:

```
wt-orchestrate --time-limit 4h start     # 4 óra limit
wt-orchestrate --time-limit none start   # nincs limit
```

Amikor a limit lejár:

1. Az állapot `time_limit`-re vált
2. Notification küldése
3. Summary email küldése
4. HTML riport generálás
5. A futás megáll, de a worktree-k megmaradnak

A futás folytatható: `wt-orchestrate start` — a timer ott folytatódik, ahol abbahagyta.

12.5 Memória és auditálás

A monitor loop 10 pollásonként (~2.5 percenként) automatikusan:

- `orch_memory_stats()`: memória rendszer állapot
- `orch_gate_stats()`: gate statisztikák
- `orch_memory_audit()`: memória audit (duplikátum detekció)

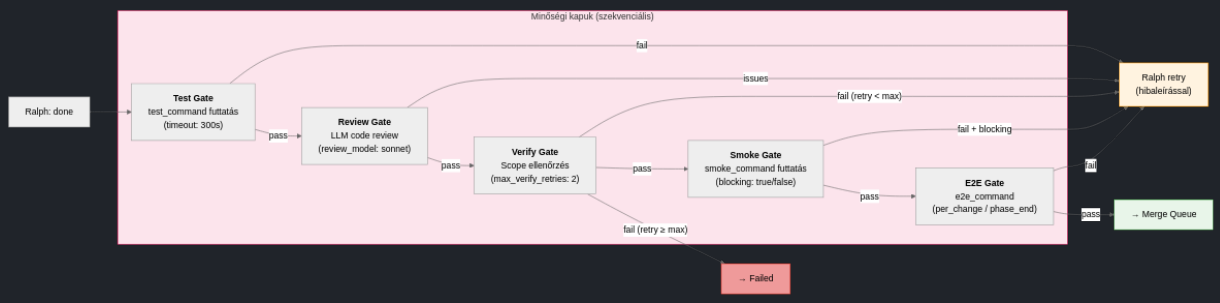
Ez biztosítja, hogy a hosszú futások során a memória rendszer egészséges marad.

13. fejezet

Minőségi Kapuk

13.1 Áttekintés

Amikor egy Ralph loop befejezi a munkáját (`loop-state.status == "done"`), a change nem kerül azonnal merge-re. Egy szekvenciális minőségi pipeline-on kell átmennie, ahol minden "gate" (kapu) ellenőrzi a munka egy aspektusát.



13.1. ábra: A minőségi kapuk szekvenciális pipeline-ja

13.2 Test Gate

Az első és legfontosabb kapu. Ha van `test_command` konfigurálva, a rendszer futtatja a teszteket a worktree-ben.

13.2.1 Működés

```
# A verifier.sh run_tests_in_worktree() függvénye:
cd <worktree_path>
timeout <test_timeout> bash -c "<test_command>"
```

13.2.2 Paraméterek

Paraméter	Alapértelmezés	Leírás
test_command	"" (kihagyás)	A futtatandó teszt parancs

Paraméter	Alapértelmezés	Leírás
test_timeout	300s	Timeout a teszt futtatásra

13.2.3 Kimenet kezelés

- **Pass** (exit code 0): Továbblépés a következő gate-re
- **Fail** (exit code nem 0): A teszt kimenet (max 2000 karakter) mentésre kerül, és a change visszakerül a Ralph loop-ba a hibaüzenettel mint retry context

A teszt statisztikák (`test_stats`) az állapotfájlba kerülnek:

```
{
  "test_result": "pass",
  "test_stats": {
    "exit_code": 0,
    "duration_ms": 12500,
    "output_tail": "Tests: 42 passed, 42 total"
  }
}
```

13.3 Review Gate

LLM-alapú code review, amelyet egy (tipikusan) kisebb és gyorsabb modell végez.

13.3.1 Működés

A rendszer:

1. Összegyűjti a change-ben végrehajtott módosításokat (`git diff`)
2. Hozzáadja a change scope-ját és a hozzárendelt követelményeket
3. Elküldi a `review_model`-nek (alapértelmezés: sonnet)
4. Az LLM értékeli a kód minőségét

13.3.2 Requirement-aware review

A `build_req_review_section()` a review prompthoz hozzáfűzi az érintett követelményeket:

```
## Assigned Requirements (this change owns these)
- REQ-001: JWT autentikáció — Token alapú auth a /api/* végpontokon
- REQ-004: Token refresh — Lejárt token automatikus frissítése

## Cross-Cutting Requirements (awareness only)
- REQ-012: Logging — Minden API hívás naplózása
```

Ez segíti az LLM-et abban, hogy ne csak a kód minőségét, hanem a **követelmény-megfelelőséget** is értékelje.

13.3.3 Konfiguráció

Paraméter	Alapértelmezés	Leírás
review_before_merge	false	Review gate engedélyezése
review_model	sonnet	Review-hoz használt model
skip_review	false	Change-szintű kihagyás (plan-ben)

13.4 Verify Gate

A verify gate ellenőrzi, hogy az implementáció a scope-nak megfelel-e.

13.4.1 Működés

A `verify_merge_scope()` függvény:

1. Összehasonlítja a tervezett scope-ot a tényleges módosításokkal
2. Ellenőrzi, hogy a lényeges változások megtörténtek-e
3. Azonosítja a nem kívánt mellékhatásokat (scope creep)

13.4.2 Retry logika

Ha a verify gate sikertelen:

```
verify_retry_count < max_verify_retries?
  → igen: retry_context + Ralph loop újraindítás
  → nem: change → failed
```

A retry context tartalmazza a verify hibaüzenetet, a build kimenetet, és az eredeti scope-ot, hogy az ágens célzottan tudja javítani a problémát.

Paraméter	Alapértelmezés	Leírás
max_verify_retries	2	Maximum retry szám

13.5 Smoke Gate

A smoke test egy könnyű, gyors ellenőrzés, amely a build integritását vizsgálja.

13.5.1 Működés

```
cd <worktree_path>
timeout <smoke_timeout> bash -c "<smoke_command>"
```

13.5.2 Blocking vs Non-blocking

Mód	Viselkedés
smoke_blocking: true	Sikertelen smoke → Ralph retry (fix ciklus)
smoke_blocking: false	Sikertelen smoke → figyelmeztetés, de merge engedélyezett

13.5.3 Smoke fix ciklus

Ha a smoke test sikertelen és `smoke_blocking: true`:

1. A hiba leírása retry context-ként átadódik
2. Ralph loop indul a smoke fix-re (korlátozott budget)
3. Max `smoke_fix_max_retries` (alapértelmezés: 3) próbálkozás
4. Ha nem sikerül → change failed

Paraméter	Alapértelmezés	Leírás
smoke_command	""	Smoke test parancs
smoke_timeout	120s	Smoke timeout
smoke_blocking	true	Blocking mód
smoke_fix_token_budget	500K	Fix budget
smoke_fix_max_turns	15	Max fix iteráció
smoke_fix_max_retries	3	Max retry

13.5.4 Health Check

Opcionálisan egy HTTP health check is konfigurálható:

```
smoke_health_check_url: "http://localhost:3000/api/health"
smoke_health_check_timeout: 30
```

A rendszer a smoke test után ellenőrzi, hogy az alkalmazás HTTP válaszol-e.

13.6 E2E Gate

A legátfogóbb teszt, amely a teljes alkalmazást vizsgálja.

13.6.1 Két mód

Mód	Leírás
e2e_mode: per_change	Minden change merge előtt fut az E2E
e2e_mode: phase_end	Az E2E csak a fázis végén fut, a main branch-en

A `phase_end` mód hatékonyabb nagy projekteknél, mert az E2E tesztek gyakran lassúak és a fő ágon futva relevánsabb eredményt adnak.

Paraméter	Alapértelmezés	Leírás
e2e_command	""	E2E teszt parancs

Paraméter	Alapértelmezés	Leírás
e2e_timeout	120s	E2E timeout
e2e_mode	per_change	per_change/phase_end

13.7 A gate-ek sorrendje összefoglalva

```
Ralph done
  → Test Gate (test_command, ha van)
    → Review Gate (review_before_merge, ha aktív)
      → Verify Gate (scope ellenőrzés)
        → Smoke Gate (smoke_command, ha van)
          → E2E Gate (e2e_command, ha van)
            → ✓ Merge Queue
```

FONTOS

Minden gate opcionális. Ha nincs test_command, a Test Gate kimarad. Ha review_before_merge false, a Review Gate kimarad. A minimális konfiguráció (semmilyen gate) esetén a change a Ralph done után közvetlenül a merge queue-ba kerül — de ez éles projekteknél nem ajánlott.

13.8 Hookak

A gate-ek között hookak futhatnak:

Hook	Mikor fut
hook_post_verify	Verify gate pass után
hook_pre_merge	Merge előtt (blokkoló)
hook_on_fail	Change failed státuszba kerüléskor

14. fejezet

Merge és Delivery

14.1 Merge Queue

Amikor egy change sikeresen átmegy minden minőségi kapun, a merge queue-ba kerül. A queue feldolgozása a `merge_policy` direktíva szerint történik.

14.2 Merge Policy

Három merge policy létezik:

14.2.1 eager

A change azonnal merge-elődik, amint a verify pipeline végére ér.

```
merge_policy: eager
```

Előny: gyors delivery, a többi worktree hamarabb kap friss main-t. Hátrány: nincs lehetőség csoportos áttekintésre.

14.2.2 checkpoint (alapértelmezett)

N db change után a rendszer megáll és emberi jóváhagyást kér.

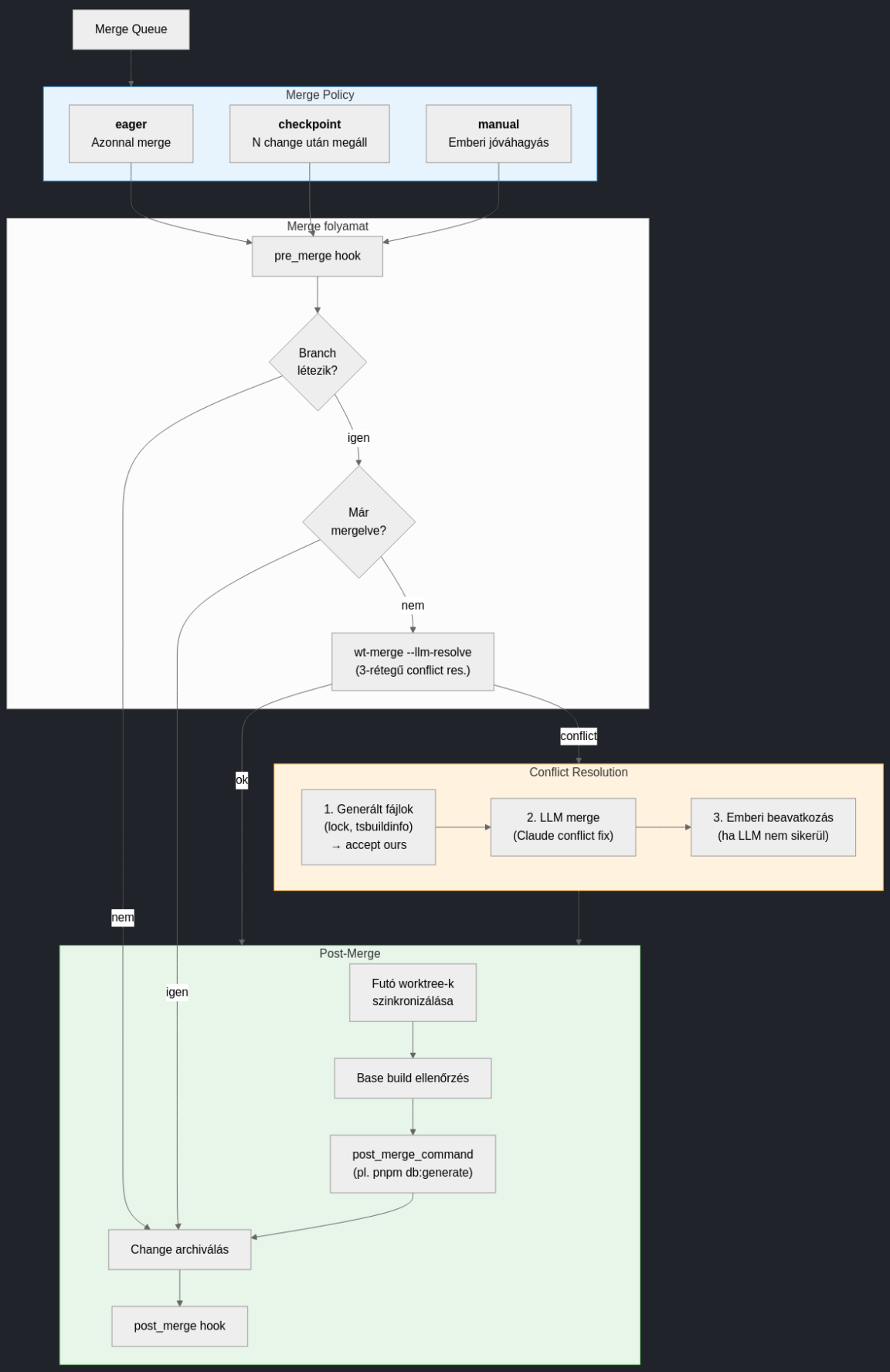
```
merge_policy: checkpoint
checkpoint_every: 3
```

Folyamat:

1. A 3. change befejeződik → checkpoint aktiválódik
2. Az orchestrátor megáll (status: `checkpoint`)
3. Az ember áttekinti a merge queue-t
4. `wt-orchestrate approve --merge` → merge + folytatás

14.2.3 manual

Minden merge emberi jóváhagyást igényel.



14.1. ábra: A merge pipeline és post-merge folyamat

```
merge_policy: manual
```

14.3 A merge folyamat

A `merge_change()` három esetet kezel:

14.3.1 Eset 1: Branch már nem létezik

Ha a `change/<name>` branch már nem létezik (valaki kézzel mergelte és törölte):

- Státusz → `merged`
- Worktree cleanup
- Change archiválás

14.3.2 Eset 2: Branch már be van mergelve

Ha a branch létezik, de már a HEAD őse:

- Státusz → `merged`
- Cleanup

14.3.3 Eset 3: Normál merge (a gyakori)

A `wt-merge` parancs végzi a tényleges merge-t:

```
wt-merge <change-name> --no-push --llm-resolve
```

14.4 3-rétegű Conflict Resolution

Merge konfliktus esetén három szinten próbál a rendszer feloldást:

14.4.1 1. réteg: Generált fájlok (automatikus)

A `sync_worktree_with_main()` és a merge logika automatikusan kezeli a generált fájlok konfliktusait:

- `package-lock.json`
- `yarn.lock`, `pnpm-lock.yaml`
- `*.tsbuildinfo`

Ezeket `--ours` stratégiával oldja fel (a worktree verziója nyer).

14.4.2 2. réteg: LLM merge (--llm-resolve)

Ha valódi kód konfliktus van, a `wt-merge --llm-resolve` egy Claude ágenst hív:

1. A conflicted fájlok átadása az LLM-nek
2. Az LLM megérti mindkét oldal szándékát
3. Az LLM össze merge-li a változtatásokat
4. Az eredmény commit-olódik

14.4.3 3. réteg: Emberi beavatkozás

Ha az LLM sem tudja feloldani a konfliktust:

- A change `merge_blocked` státuszba kerül
- Notification küldése
- Az ember kézi merge-t végez
- Az orchestrátor a következő poll-ban észleli az új állapotot

14.5 Post-Merge Pipeline

Sikeres merge után a következő lépések futnak:

14.5.1 1. Futó worktree-k szinkronizálása

A `_sync_running_worktrees()` biztosítja, hogy a többi aktív worktree megkapja a friss main-t:

```
main ← merge(auth-system)
  ↓ sync
  worktree/user-profile ← git merge main
  worktree/api-endpoints ← git merge main
```

Ez megakadályozza, hogy a többi ágens elavult kódra építsen.

14.5.2 2. Base build ellenőrzés

A merge után a base build cache érvénytelenítődik (`BASE_BUILD_STATUS=""`), és a következő verify gate újraellenőrzi, hogy a main branch buildelhető-e.

14.5.3 3. Post-merge command

Ha van `post_merge_command` konfigurálva:

```yaml
post_merge_command: "pnpm db:generate"
```

Ez a parancs a merge után fut le a projekt gyökerében. Tipikus használat:

- Adatbázis migráció generálás (`prisma generate`, `drizzle-kit generate`)
- Build artifact frissítés
- Cache invalidálás

14.5.4 4. Change archiválás

A `archive_change()` az OpenSpec change könyvtárat az archívumba mozgatja:

```
openspec/changes/auth-system/
  → openspec/changes/archive/2026-03-10-auth-system/
```

14.5.5 5. Post-merge hook

Ha van `hook_post_merge` konfigurálva, az lefut a change nevével mint argumentum.

14.5.6 6. Coverage frissítés

A `update_coverage_status()` frissíti a requirement lefedettséget: a change-hez rendelt REQ-XXX azonosítók `merged` státuszba kerülnek.

14.6 Checkpoint és Approval

14.6.1 Checkpoint aktiválás

Checkpoint az alábbi esetekben aktiválódik:

Trigger	Leírás
`checkpoint_every: N`	N merge után
`token_hard_limit`	Token hard limit elérése
Manuális	`wt-orchestrate pause --all`

14.6.2 Approval folyamat

```
wt-orchestrate approve           # jóváhagyás, folytatás
wt-orchestrate approve --merge   # jóváhagyás + azonnali merge flush
```

A `--merge` flag a merge queue-ban várakozó change-eket azonnal merge-li.

14.6.3 Auto-approve (E2E futtatásokhoz)

```yaml
checkpoint_auto_approve: true
```

Ez felügyelet nélküli futtatásokhoz használatos (pl. CI/CD, E2E tesztelés).

> **FONTOS**
>
> A merge policy a projekthez és a csapat igényeihez igazítandó. Kis projekteknél az eager policy a leghatékonyabb. Nagyobb projekteknél a checkpoint policy biztosítja, hogy az ember lássa az eredményt, mielőtt túl sok change merge-elődne.

14.7 Merge retry intelligencia

Ha egy merge sikertelen (pl. build hiba merge után), a rendszer:

1. A build kimenetet menti (`build_output`)
2. A change-et `merge_blocked` státuszba rakja
3. A következő poll-ban újrapróbálja
4. Ha a `fix_base_build_with_llm()` elérhető, LLM-mel próbálja javítani a hibát

15. fejezet

Tapasztalatok és kihívások

15.1 Megoldott problémák

Az alábbiak éles orchestrációs futtatások során derültek ki — jellemzően az első néhány production run (sales-raketa, MiniShop, CraftBrew) tanulságai.

15.1.1 Token budget kalibráció

Probléma: Az eredeti budget tierek (S=500K, M=2M) túl alacsonyak voltak. Az artifact-generálási overhead (proposal → design → specs → tasks) már 500K-800K tokent igényel, mielőtt az implementáció elkezdődne.

Megoldás: Kalibrálás éles adatokból: S=2M, M=5M, L=10M. Az éles futtatások visszaigazolták: egy tipikus S change 1.3-1.5M tokent használ (artifact overhead-del együtt).

15.1.2 Watchdog spam

Probléma: A watchdog 15 másodpercenként emittált `WATCHDOG_WARN` eseményeket — egy lassú change-nél 60+ bejegyzés az event log-ban, ami rontotta a sentinel élőség-detekciót és felfújta a napló méretét.

Megoldás: Throttle az event emissziónál: csak minden 20. előfordulásnál ír a naplóba.

15.1.3 Sentinel fájlnév eltérés

Probléma: A sentinel a `orchestration-events.jsonl` fájlt figyelte (hardcoded), de az orchestrátor `orchestration-state-events.jsonl`-be írt (dinamikus név). Eredmény: a sentinel 180 másodperc után megölte az egészséges orchestrátort.

Megoldás: A sentinel a state fájlnévből származtatja az events fájlnevet, nem hardcode-olja.

15.1.4 Token tracking félrevezető adatok

Probléma: A cache tokenek beleszámítottak a budget-be, de nem jelentek meg a tracking-ben — 18x eltérés a kijelzett és a tényleges fogyasztás között.

Megoldás: Cache tokenek bevétele a tracking-be.

15.1.5 Nagy change-ek megbízhatósága

Probléma: 14+ requirement egy change-ben → 40+ perc, 3 verify retry → failed. Az ok: a kontextus ablak nem elég nagy, az ágens elveszíti a fonalat.

Megoldás: A planner max 6 REQ/change szabályt alkalmaz. 4-6 requirement-es change-ek megbízhatóan futnak (12-19 perc, 0 retry).

15.1.6 Jest + Playwright ütközés

Probléma: A Jest unit tesztek crash-eltek a Playwright `.spec.ts` fájlokon ("TypeError: Class extends value undefined"), mert a jsdom nem kezeli a browser importokat.

Megoldás: `testPathIgnorePatterns` a jest config-ban + worktree-nkénti port izoláció a Playwright tesztekhez.

15.2 Ismert limitációk

15.2.1 Mock-alapú tesztek elrejtik a runtime hibákat

Probléma: A Jest tesztek mock-olják a Next.js API-kat (`cookies()`, `headers()`), így a runtime hibákat nem kapják el. A MiniShop futtatásban 81 Jest teszt zöld volt, miközben az alkalmazás 3 kritikus runtime hibával rendelkezett (auth bypass, cookie crash, dead link).

Tanulság: A smoke test (`pnpm build && pnpm test`) és az E2E teszt (`Playwright`) együtt szükséges. A build elkapja a type hibákat, az E2E a funkcionális regressziókat.

15.2.2 Megosztott erőforrások merge konfliktusa

Probléma: Párhuzamos change-ek ugyanazt a fájlt módosítják (pl. `functional-conventions.md`, layout komponensek). Az LLM merge 900+ soros konfliktust nem tud feloldani.

Jövő: A planner-nek fel kellene ismernie a cross-cutting fájlokat és sorrendbe állítani a change-eket, nem párhuzamosítani.

15.2.3 Kontextusvesztés merge után

Probléma: Schema változás merge után: a worktree-ben a kód típushelyesen fordult, de a main-en (ahol más change-ek is megváltoztatták a schemát) TypeScript hiba keletkezett.

Tanulság: A post-merge build ellenőrzés (`base build health check`) nélkülözhetetlen.

15.2.4 Memória nem garantálja a minőséget

Probléma: A benchmark futtatások kimutatták, hogy a memóriával rendelkező ágensek 50%-kal kevesebbet kutatnak, de alacsonyabb minőségű kódot adnak — a memória "shortcut"-ot biztosít, ami megkerüli a szükséges kód-olvasást.

Tanulság: “Recall-then-verify” minta: a memória lekérdezés után mindig ellenőrizni kell a kódbázis aktuális állapotát.

15.3 Ami még hátra van

15.3.1 Cascade failure logika

Ha egy change elbukik, a függő change-ek jelenleg örökre pending-ben maradnak. Szükséges: automatikus cascade failure propagálás, amely a függő change-eket `skipped` státuszba állítja.

15.3.2 Trend-alapú token budget

A statikus budget limetek (S=2M, M=5M) nem skálázódnak minden projektre. Szükséges: projektszintű tanulás az aktuális token-fogyasztásból, automatikus limit-igazítás.

15.3.3 Ágens-pontozás dispatch-hoz

Jelenleg a dispatch round-robin módon történik. Szükséges: ágensek pontozása hibaráta, token-hatékonyság és iteráció/haladás arány alapján — a jobb ágensek több feladatot kapnak.

15.3.4 Circuit breaker API hívásokhoz

Egymás utáni API hibák esetén a rendszer nem áll le, hanem továbbra is próbálkozik. Szükséges: circuit breaker pattern, amely N egymást követő hiba után leáll.

FONTOS

Az orchestráció értéke a második futtatástól kezdve jelentkezik igazán. Az első futtatás mindig felfed problémákat — rossz budget, hiányzó teszt konfiguráció, váratlan merge konfliktus. A lényeg: minden hiba egyszer fordul elő, mert a rendszer tanul belőle (watchdog tuning, budget kalibráció, planner szabályok).

16. fejezet

Replan és Coverage

16.1 Auto-Replan

Az auto-replan a többfázisú specifikációk kezelésére szolgál. Ha az összes change egy fázisban befejeződött, a rendszer automatikusan megtervezi és elindítja a következő fázist.

16.1.1 Aktiválás

```
auto_replan: true
```

16.1.2 Folyamat

1. A monitor loop észleli, hogy minden change `merged` vagy `failed`
2. Ha `auto_replan: true` → `cmd_replan()` hívás
3. A planner megkapja a frissített specifikációt + a kész fázisok listáját
4. Új plan generálódik a következő fázisra
5. A korábbi token számlálók mentésre kerülnek (`prev_total_tokens`)
6. Az állapot resetelődik az új plan-nel
7. A monitor loop folytatja a végrehajtást

16.1.3 Retry logika

Ha a replan sikertelen (pl. az LLM nem tud jó plan-t generálni):

- Max `MAX_REPLAN_RETRIES` (3) egymás utáni próbálkozás
- Sikertelen replan → figyelmeztetés + várakozás a következő poll-ra
- Ha 3-szor is sikertelen → az orchestrátor megáll

16.1.4 All-done detekció

Az orchestrátor “kész”-nek tekinti a futást, ha:

1. Nincs `pending` vagy `running` change
2. Nincs merge queue elem
3. `auto_replan` false VAGY a replan “nincs több fázis”-t jelent

```
Fázis 1: [A, B, C] → merged → replan
Fázis 2: [D, E]    → merged → replan
Fázis 3: nincs több → done
```

16.2 Manuális Replan

A felhasználó bármikor kérhet replant:

```
wt-orchestrate replan
```

Ez hasznos, ha:

- A specifikáció megváltozott futás közben
- A felhasználó új feladatokat adott hozzá
- Néhány change failed és új megközelítés szükséges

A manuális replan:

1. Megőrzi a `merged` change-ek állapotát
2. Figyelembe veszi a `failed` change-eket
3. Csak az új/módosított feladatokra generál change-eket

16.3 Requirement Coverage

A coverage rendszer a digest-ből származó REQ-XXX azonosítókon alapul. Célja: biztosítani, hogy a specifikáció minden követelménye le legyen fedve implementációval.

16.3.1 Coverage riport

```
wt-orchestrate coverage
```

Kimenet:

```
── Requirement Coverage ──

Coverage: 12/15 (80.0%)

Status breakdown:
  ✓ merged:     8
  ◐ running:    3
  ○ pending:    1
  ✗ failed:     1
  · unassigned: 2

Details:
  REQ-001  ✓  JWT autentikáció          → auth-system (merged)
  REQ-002  ✓  User profil               → user-profile (merged)
  REQ-003  ◐  Avatar feltöltés          → user-profile (running)
  REQ-004  ✓  Token refresh             → auth-system (merged)
  ...
```

```
  REQ-014  ·  Admin audit log          → (not assigned)
  REQ-015  ·  Export funkció           → (not assigned)
```

16.3.2 Coverage tracking lifecycle

```
Plan generálás:
  change.requirements = ["REQ-001", "REQ-004"]
  → populate_coverage() → coverage state init

Change végrehajtás közben:
  status = "running" → REQ-001 status = "running"

Change merge:
  update_coverage_status("auth-system", "merged")
  → REQ-001 status = "merged"
  → REQ-004 status = "merged"

Change failed:
  → REQ-XXX status = "failed"
  → check_coverage_gaps(): figyelmeztetés

Replan:
  A gap-ek (unassigned, failed) bekerülnek a planner prompt-ba
```

16.3.3 Coverage gap detekció

A `check_coverage_gaps()` figyeli:

- **Unassigned**: Egy REQ-XXX-hez nincs change rendelve
- **Failed**: A change, amelyhez a requirement tartozik, elbukott
- **Orphaned**: A requirement a digest-ben van, de a plan-ből hiányzik

Ezeket a gap-eket a replan figyelembe veszi és új change-eket generál rájuk.

16.3.4 Cross-cutting követelmények

Egyes követelmények több change-et is érinthetnek. Az `also_affects_reqs` mező ezt jelzi:

```
{
  "name": "api-endpoints",
  "requirements": ["REQ-005"],
  "also_affects_reqs": ["REQ-012"]
}
```

REQ-012 (pl. “Logging”) nem az `api-endpoints` fő feladata, de érintett. A review gate ezt figyelembe veszi az értékelésben.

16.4 Final Coverage Check

A futás végén (done, time-limit) a `final_coverage_check()` egy összesítő riportot generál:

```
Final Coverage: 14/15 (93.3%)
  Uncovered: REQ-015 (Export funkció)
```

Ez az összegzés bekerül:

- A summary email-be
- A HTML riportba
- Az orchestration-summary.md fájlba

FONTOS

A coverage tracking nem garancia a helyes implementációra — csak azt jelzi, hogy egy change "megpróbálta" lefedni a követelményt. A tényleges minőséget a verify és review gate-ek biztosítják. A coverage rendszer fő értéke: a replan nem felejt el semmit a specifikációból.

16.5 Összefoglaló riportok

16.5.1 HTML riport

A `generate_report()` (a `reporter.sh`-ban) egy részletes HTML riportot generál:

- Change-enkénti összefoglaló (státusz, token használat, idő)
- Coverage mátrix
- Gate statisztikák (pass/fail arányok)
- Timeline vizualizáció

16.5.2 Summary email

Ha email notifikáció konfigurálva van, a futás végén összegző email küldése:

- Futási idő (aktív és fali)
- Change eredmények
- Coverage összefoglaló
- Következő lépések (ha replan aktív)

16.5.3 orchestration-summary.md

Ember által olvasható markdown összefoglaló, amely a futás végén generálódik.

17. fejezet

Hivatkozások

17.1 CLI Referencia

17.1.1 wt-orchestrate

A fő orchestrációs parancs.

```
wt-orchestrate [globális opciók] <parancs> [parancs opciók]
```

17.1.1.1 Globális opciók

Opció	Leírás
--spec <path>	Specifikáció fájl vagy könyvtár
--brief <path>	Brief fájl (legacy)
--phase <hint>	Fázis szűrő (--spec-hez)
--max-parallel <N>	Max párhuzamos override
--time-limit <dur>	Időlimit (pl. ``4h'', ``2h30m'', ``none'')

17.1.1.2 Parancsok

Parancs	Leírás
plan [--show]	Plan generálás vagy meglévő megjelenítés
start	Plan végrehajtás indítása
status	Aktuális állapot megjelenítés
pause <name|--all>	Change vagy összes felfüggesztése
resume <name|--all>	Change vagy összes folytatása
replan	Újratervezés frissített spec-ből
approve [--merge]	Checkpoint jóváhagyás
digest --spec <path>	Strukturált digest generálás
coverage	Requirement lefedettség riport
specs [list|show|archive]	Spec dokumentumok kezelése
events [opts]	Eseménynapló lekérdezés

Parancs	Leírás
tui	Terminál dashboard indítás
self-test	Belső tesztek futtatása

17.1.2 wt-new / wt-merge / wt-loop

Parancs	Leírás
wt-new <name>	Új worktree és branch létrehozás
wt-merge <name> [--llm-resolve]	Worktree merge main-be
wt-loop start [opts]	Ralph loop indítás
wt-loop status	Loop állapot lekérdezés

17.2 Change állapotgép

A change-ek a következő állapotok között mozognak az életciklusuk során:

17.1. ábra: A change teljes állapotgépe

17.2.1 Állapotok

Állapot	Leírás
pending	Várakozik dispatch-re (függőségek nem teljesültek)
dispatched	Worktree létrehozva, Ralph loop indítás alatt
running	Ralph loop aktív, fejlesztés folyamatban
verifying	Verify pipeline fut (test → review → verify → smoke)
verify-failed	Verify gate sikertelen, retry várakozik
stalled	Watchdog észlelte, hogy az ágens elakadt
paused	Manuálisan felfüggesztve
waiting:budget	Token budget limit miatt várakozik
budget_exceeded	Token budget meghaladva
merge_queue	Verify sikeres, merge-re vár
merging	Merge folyamatban
merge_blocked	Merge konfliktus, nem tudta feloldani
merged	Sikeresen merge-elve a fő ágba
smoking	Post-merge smoke test fut
smoke_failed	Smoke test sikertelen
smoke_blocked	Smoke fix retry limit elérve
completed	Minden gate sikeresen teljesítve
failed	Véglegesen sikertelen (retry limit elérve)
skipped	Kihagyva (függőség failed)

17.2.2 Főbb állapotátmenetek

```
pending → dispatched → running → verifying
  ↑                       ↕          ↓
  │                   stalled    merge_queue → merging → merged → completed
  │                       ↓          ↑
  │                   failed    verify-failed (→ running retry)
  │                                  ↓
  └── skipped (dependency failed)  failed (retry exhausted)
```

17.3 Direktíva táblázat

A direktívák a rendszer viselkedését szabályozzák. Az alábbi táblázatok kategóriánként mutatják az összes elérhető beállítást. A legtöbb projekt az alapértelmezett értékekkel indul, és csak a tesztelési parancsokat (`test_command`, `smoke_command`) és a párhuzamossági szintet (`max_parallel`) állítja be. A többi direktíva finomhangolásra szolgál, amire jellemzően csak az első néhány futtatás tapasztalatai után van szükség.

17.3.1 Végrehajtás

Ezek a direktívák a futtatás alapvető paramétereit határozzák meg: hány ágens dolgozzon egyszerre, hogyan történjen a merge, és meddig futhat a rendszer.

Direktíva	Típus	Alapért.	Leírás
max_parallel	szám	3	Max párhuzamos worktree
merge_policy	enum	checkpoint	eager / checkpoint / manual
checkpoint_every	szám	3	Checkpoint merge szám
time_limit	string	5h	Aktív idő limit
pause_on_exit	bool	false	Pause ha orchestrátor leáll
context_pruning	bool	true	Context window optimalizálás
model_routing	string	off	Model routing stratégia

17.3.2 Modellek

Három modell-szint van: opus a nehéz munkához (implementáció), sonnet a gyorsabb feladatokhoz (review, egyszerű change-ek), haiku az összegzéshez. A model routing automatikusan választhat a change komplexitása alapján.

Direktíva	Típus	Alapért.	Leírás
default_model	string	opus	Implementáció model
review_model	string	sonnet	Review model
summarize_model	string	haiku	Összegzés model

17.3.3 Tesztelés

A tesztelési direktívák a minőségi gate-eket konfigurálják. A `test_command` a legfontosabb: ha üres, a teszt gate teljesen kimarad. A smoke és E2E tesztek opcionálisak, de éles futtatásnál erősen ajánlottak — a smoke elkapja a build hibákat, az E2E a funkcionális regressziókat.

Direktíva	Típus	Alapért.	Leírás
test_command	string	""	Teszt parancs
test_timeout	szám	300	Teszt timeout (mp)
smoke_command	string	""	Smoke test parancs
smoke_timeout	szám	120	Smoke timeout (mp)
smoke_blocking	bool	true	Smoke fail blokkol-e
smoke_fix_token_budget	szám	500K	Smoke fix token limit
smoke_fix_max_turns	szám	15	Smoke fix max iteráció
smoke_fix_max_retries	szám	3	Smoke fix max retry
smoke_health_check_url	string	""	Health check URL
smoke_health_check_timeout	szám	30	Health check timeout
e2e_command	string	""	E2E test parancs
e2e_timeout	szám	120	E2E timeout (mp)
e2e_mode	enum	per_change	per_change / phase_end

17.3.4 Review és Verify

Direktíva	Típus	Alapért.	Leírás
review_before_merge	bool	false	LLM review gate
max_verify_retries	szám	2	Verify retry limit

17.3.5 Token kontroll

A token kontroll a költségek kezelésére szolgál. A soft limit (`token_budget`) figyelmeztet és lelassítja a dispatch-et, de a futó ágensek befejezhetik a munkájukat. A hard limit (`token_hard_limit`) megállítja a rendszert és emberi jóváhagyást kér — ez a végső biztonsági háló a váratlan tokenfogyasztás ellen.

Direktíva	Típus	Alapért.	Leírás
token_budget	szám	0	Soft limit (0=ki)
token_hard_limit	szám	20M	Hard limit
checkpoint_auto_approve	bool	false	Auto checkpoint approve

17.3.6 Tervezés

Direktíva	Típus	Alapért.	Leírás
auto_replan	bool	false	Auto-replan fázis végén
plan_method	enum	api	api / agent
plan_token_budget	szám	500K	Agent plan budget

17.3.7 Watchdog

A watchdog az "éjszakai őr": akkor lép közbe, amikor az ágens elakad és magától nem tud továbblépni. A timeout értéket érdemes a projekt build idejéhez igazítani — ha a build 5 percet vesz igénybe, a 600s-os timeout helyes; ha 30 másodpercet, érdemes levinni 180s-ra.

Direktíva	Típus	Alapért.	Leírás
watchdog_timeout	szám	600	Stall timeout (mp)
watchdog_loop_threshold	szám	5	Loop detekció küszöb
max_redispatch	szám	2	Max redispatch próba

17.3.8 Hookak

A hookak lehetővé teszik, hogy a projekt-specifikus logika beépüljön a pipeline-ba. A `pre_merge` hook blokkoló: ha hibát dob, a merge nem történik meg. A többi hook nem blokkoló — ha hibáznak, a pipeline folytatódik. A `post_merge_command` a leggyakrabban használt hook: tipikusan adatbázis migrációs generálásra (Prisma, Drizzle) vagy build artifact frissítésre szolgál.

Direktíva	Típus	Alapért.	Leírás
hook_pre_dispatch	string	""	Dispatch előtt

Direktíva	Típus	Alapért.	Leírás
hook_post_verify	string	""	Verify után
hook_pre_merge	string	""	Merge előtt (blokkoló)
hook_post_merge	string	""	Merge után
hook_on_fail	string	""	Fail-kor
post_merge_command	string	""	Post-merge parancs

17.3.9 Eseménynapló

Direktíva	Típus	Alapért.	Leírás
events_log	bool	true	Eseménynapló aktív-e
events_max_size	szám	1MB	Max napló méret

17.4 Fájl struktúra referencia

17.4.1 Projekt szintű fájlok

```
.claude/
├── orchestration.yaml          ← konfiguráció
└── orchestration.log           ← futási napló

wt/orchestration/
├── digest/
│   ├── requirements.json       ← REQ-XXX azonosítók
│   ├── phases.json             ← fázis struktúra
│   ├── digest-meta.json        ← hash, dátum
│   └── ambiguities.json        ← kétértelmű pontok
└── specs/
    ├── v12.md                  ← aktív spec-ek
    └── archive/
        └── v11.md              ← archivált spec-ek

orchestration-plan.json         ← aktuális plan (gitignore)
orchestration-state.json        ← futási állapot (gitignore)
orchestration-summary.md        ← összefoglaló (gitignore)
```

17.4.2 Worktree szintű fájlok

```
.claude/worktrees/<change-name>/
├── .claude/
│   └── loop-state.json         ← Ralph loop állapot
├── openspec/changes/<name>/
│   ├── proposal.md             ← OpenSpec proposal
│   ├── design.md               ← tervezési dokumentum
```

```
│   └── tasks.md                ← feladatlista
└── ... (projekt fájlok)
```

17.4.3 Eseménytípusok

Esemény	Leírás
STATE_CHANGE	Change státusz változás
DISPATCH	Change dispatch
MERGE_ATTEMPT	Merge kísérlet
MERGE_SUCCESS	Sikeres merge
MERGE_FAIL	Sikertelen merge
VERIFY_PASS	Verify gate pass
VERIFY_FAIL	Verify gate fail
TEST_PASS / TEST_FAIL	Teszt eredmény
SMOKE_PASS / SMOKE_FAIL	Smoke teszt eredmény
WATCHDOG_ESCALATE	Watchdog eszkaláció
CHECKPOINT	Checkpoint aktiválás
REPLAN	Replan indítás
DIGEST_STARTED / DIGEST_FAILED	Digest események

Az események a `.claude/orchestration.log` JSONL fájlba kerülnek, és a `wt-orchestrate events` paranccsal lekérdezhetők:

```
wt-orchestrate events --type MERGE_SUCCESS --last 10 --json
```